MOT LOGISTICS

TRANSPORT TRIP MANAGEMENT & AUTOMATED MONTHLY INVOICE SYSTEM

Formerly SBP Kubre Tempo Service | Multi-Tenant Logistics Engine

Core System Source Code Repository

Official Technical Documentation & Core Function Source Code Ledger

Application Name:	MOT Logistics (Transport Management & Automated Invoicing)
Core Language & Runtime:	Java 17 (Pure Core Java Architecture - Zero External PDF Libs)
Database Management:	MySQL 8.0 Relational RDBMS via JDBC Connection Pool
Tenancy Architecture:	Multi-Tenant Enterprise Isolation (Om Sai, Oracle, Company XYZ)
Core Generation Engine:	Native Core Java PDF 1.4 Binary Generator (Invoices, Receipts, Audits)
Security & Validation:	Role-Based Auth (Admin vs Driver), Parametrized PreparedStatements
Total Core Modules:	20 Production Modules (~3,700 Lines of Core Source Code)
Document Volume:	47 Pages (Compliant with <= 50 Page Requirement)

ARCHITECTURAL OVERVIEW & SUBSYSTEMS

1. Multi-Tenant Company Isolation

Independent admin accounts per enterprise client. Companies only access their own transport records and invoices with zero data leakage.

2. Driver Fleet & Vehicle Operations

Independent driver logins with multi-vehicle registration. Drivers view all trips across all clients with unified earnings.

3. Automated Monthly Invoice Engine

Aggregates completed trips by client/month, computes gross receivables, handles partial settlements, and produces official invoices.

4. Pure Core Java PDF 1.4 Generation

Direct binary PDF stream generator without heavy third-party dependencies (iText/PDFBox). Generates Tax Invoices, Receipts, and Reports.

5. Live Live Connection & Driver Attribution

Trips submitted in driver portal instantly reflect in real-time across company rosters and ledgers with full driver attribution.

CONFIDENTIAL & PROPRIETARY

This technical document contains source code for the MOT Logistics Transport Management & Invoice System.
Generated automatically on November 9, 2026 for system audit, backup, and code inspection.

TABLE OF CONTENTS - CORE SOURCE CODE FILES

20 Core Files | 47 Total Pages

#	File Name	Category	Module Function & Purpose	Page
1	`Main.java`	[APPLICATION]	Application Entry Point & Lifecycle Bootstrap	3
2	`schema.sql`	[DATABASE]	Relational Schema DDL & Multi-Tenant Constraints	3
3	`DatabaseConnection.java`	[DATABASE]	MySQL Singleton Connection & Dynamic Environment Config	5
4	`Trip.java`	[MODEL]	Trip Domain Model & Financial Transit Entity	6
5	`Invoice.java`	[MODEL]	Monthly Billing & Transport Invoice Domain Model	7
6	`Driver.java`	[MODEL]	Driver & Vehicle Owner Identity Model	8
7	`Company.java`	[MODEL]	Enterprise Client Company Tenant Model	9
8	`Payment.java`	[MODEL]	Financial Settlement & Receivables Model	10
9	`Vehicle.java`	[MODEL]	Commercial Fleet Vehicle Asset Model	11
10	`Admin.java`	[MODEL]	Company Admin Account & Security Model	11
11	`TripDAO.java`	[DAO]	Trip Data Access, Tenant Isolation, Cross-Company Queri...	12
12	`InvoiceDAO.java`	[DAO]	Automated Monthly Invoice Calculation, Ledger & Archiva...	18
13	`DriverDAO.java`	[DAO]	Driver Authentication, Multi-Vehicle Operations & Compa...	21
14	`CompanyDAO.java`	[DAO]	Company Tenant Auth, Security & Financial Dashboards DAO	26
15	`PaymentDAO.java`	[DAO]	Payment Processing, Partial Settlements & Balance Track...	27
16	`VehicleDAO.java`	[DAO]	Fleet Vehicle Registration & Assignment DAO	29
17	`AdminDAO.java`	[DAO]	System Admin Authentication & Recovery DAO	32
18	`InvoiceGenerator.java`	[ENGINE]	Core Java PDF 1.4 Binary Generator (Invoices, Receipts ...	34
19	`Validation.java`	[UTIL]	Input Sanitization, Mobile, Email & Vehicle Plate Valid...	43
20	`TripEntryPanel.java`	[UI/WORKFLOW]	Driver Trip Logging Form & Instant Company Connection E...	44

** Note: Each code file is printed with exact line numbers and clean indentation. Page references indicate starting page.*

FILE: src/com/transport/Main.java **[APPLICATION] | 51 lines**

Application Entry Point & Lifecycle Bootstrap

```
1  | package com.transport;
2  |
3  | import com.transport.db.DatabaseConnection;
4  | import com.transport.ui.LoginFrame;
5  |
6  | import javax.swing.*;
7  |
8  | /**
9  |  * Main.java - Application Entry Point for MOT Logistics
10 |  */
11 | public class Main {
12 |
13 |     public static void main(String[] args) {
14 |
15 |         System.out.println("=====================================================");
16 |         System.out.println("     MOT LOGISTICS - Starting Desktop System...     ");
17 |         System.out.println("=====================================================");
18 |
19 |         // Pre-flight database connectivity check
20 |         System.out.print("Connecting to MySQL Database 'transport_db'... ");
21 |         if (DatabaseConnection.testConnection()) {
22 |             System.out.println("ONLINE & READY!");
23 |         } else {
24 |             System.out.println("OFFLINE / FAILED!");
25 |             JOptionPane.showMessageDialog(null,
26 |                 "Unable to connect to MySQL database 'transport_db'!\n\n" +
27 |                 "Troubleshooting Steps:\n" +
28 |                 "1. Confirm MySQL server is running (e.g. brew services start mysql)\n" +
29 |                 "2. Ensure database 'transport_db' is imported (mysql -u root < database/sche...
30 |                 "3. Check password in DatabaseConnection.java (default is blank)\n" +
31 |                 "4. Verify lib/mysql-connector-j.jar is present",
32 |                 "MOT Logistics - Database Error",
33 |                 JOptionPane.ERROR_MESSAGE);
34 |             System.exit(1);
35 |         }
36 |
37 |         // Apply clean Look and Feel
38 |         try {
39 |             UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
40 |         } catch (Exception e) {
41 |             System.out.println("Using default Swing Look and Feel: " + e.getMessage());
42 |         }
43 |
44 |         // Launch UI on Event Dispatch Thread
45 |         SwingUtilities.invokeLater(() -> {
46 |             LoginFrame loginFrame = new LoginFrame();
47 |             loginFrame.setVisible(true);
48 |             System.out.println("MOT Logistics UI successfully initialized.");
49 |         });
50 |     }
51 | }
```

FILE: database/schema.sql **[DATABASE] | 140 lines**

Relational Schema DDL & Multi-Tenant Constraints

```
1  | -- ============================================================
2  | -- MOT LOGISTICS - Transport Management & Automated Invoice System
3  | -- Production-Grade Multi-Tenant Schema & Realistic Simulation Data
4  | -- ============================================================
5  |
6  | CREATE DATABASE IF NOT EXISTS transport_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
7  | USE transport_db;
8  |
9  | SET NAMES utf8mb4;
10 | SET FOREIGN_KEY_CHECKS = 0;
11 |
12 | -- ------------------------------------------------------------
13 | -- Drop existing tables to establish clean updated relational structure
14 | -- ------------------------------------------------------------
15 | DROP TABLE IF EXISTS payment;
16 | DROP TABLE IF EXISTS invoice;
17 | DROP TABLE IF EXISTS trip;
18 | DROP TABLE IF EXISTS vehicle;
19 | DROP TABLE IF EXISTS driver;
20 | DROP TABLE IF EXISTS company;
21 | DROP TABLE IF EXISTS admin;
22 |
23 | -- ------------------------------------------------------------
24 | -- Table: company (Enterprise Transport Client Tenants)
25 | -- Each company has its own independent admin login and workspace
26 | -- ------------------------------------------------------------
27 | CREATE TABLE company (
28 |     id              INT AUTO_INCREMENT PRIMARY KEY,
29 |     company_name    VARCHAR(100) NOT NULL UNIQUE,
```

```
    admin_name      VARCHAR(100) NOT NULL,
    email           VARCHAR(100) NOT NULL UNIQUE,
    username        VARCHAR(50)  NOT NULL UNIQUE,
    password        VARCHAR(100) NOT NULL,
    contact_number  VARCHAR(20),
    address         VARCHAR(200),
    gst_number      VARCHAR(20),
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ------------------------------------------------------------
-- Table: driver (Professional Commercial Drivers / Fleet Operators)
-- Each driver has an individual login account and vehicle assignment
-- ------------------------------------------------------------
CREATE TABLE driver (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    driver_name     VARCHAR(100) NOT NULL,
    mobile_number   VARCHAR(20)  NOT NULL,
    email           VARCHAR(100),
    licence_number  VARCHAR(30)  NOT NULL,
    username        VARCHAR(50)  NOT NULL UNIQUE,
    password        VARCHAR(100) NOT NULL,
    is_active       TINYINT(1)   DEFAULT 1,
    company_id      INT DEFAULT NULL,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (company_id) REFERENCES company(id) ON DELETE SET NULL
);

-- ------------------------------------------------------------
-- Table: vehicle (Commercial Logistics Fleet)
-- Commercial goods carriers operated by licensed drivers
-- ------------------------------------------------------------
CREATE TABLE vehicle (
    id                  INT AUTO_INCREMENT PRIMARY KEY,
    registration_number VARCHAR(20) UNIQUE NOT NULL,
    vehicle_type        VARCHAR(50) NOT NULL,
    driver_id           INT,
    status              VARCHAR(20) DEFAULT 'Active',
    FOREIGN KEY (driver_id) REFERENCES driver(id) ON DELETE SET NULL
);

-- ------------------------------------------------------------
-- Table: trip (Operational Cargo Movements & Consignments)
-- Multi-tenant isolation by company_id and driver_id
-- ------------------------------------------------------------
CREATE TABLE trip (
    id                  INT AUTO_INCREMENT PRIMARY KEY,
    trip_code           VARCHAR(20) UNIQUE NOT NULL,
    trip_date           DATE NOT NULL,
    company_id          INT NOT NULL,
    driver_id           INT NOT NULL,
    vehicle_id          INT NOT NULL,
    source_location     VARCHAR(100) NOT NULL,
    destination         VARCHAR(100) NOT NULL,
    goods_description   VARCHAR(200),
    amount              DECIMAL(10,2) NOT NULL,
    trip_status         VARCHAR(20) DEFAULT 'Completed',
    remarks             VARCHAR(200),
    created_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (company_id) REFERENCES company(id) ON DELETE CASCADE,
    FOREIGN KEY (driver_id)  REFERENCES driver(id)  ON DELETE RESTRICT,
    FOREIGN KEY (vehicle_id) REFERENCES vehicle(id) ON DELETE RESTRICT
);

-- ------------------------------------------------------------
-- Table: invoice (Monthly & Cumulative Billing Invoices)
-- Scoped for Company client billing and Driver earnings vouchers
-- ------------------------------------------------------------
CREATE TABLE invoice (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    invoice_number  VARCHAR(20) UNIQUE NOT NULL,
    invoice_date    DATE NOT NULL,
    company_id      INT NOT NULL,
    driver_id       INT,
    vehicle_id      INT,
    billing_month   INT NOT NULL,
    billing_year    INT NOT NULL,
    total_trips     INT DEFAULT 0,
    total_amount    DECIMAL(10,2) DEFAULT 0.00,
    amount_paid     DECIMAL(10,2) DEFAULT 0.00,
    remaining_amount DECIMAL(10,2) DEFAULT 0.00,
    invoice_type    VARCHAR(20) DEFAULT 'Company',
    status          VARCHAR(20) DEFAULT 'Generated',
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (company_id) REFERENCES company(id) ON DELETE CASCADE,
    FOREIGN KEY (driver_id)  REFERENCES driver(id)  ON DELETE SET NULL
);

-- ------------------------------------------------------------
```

```
119 |  -- Table: payment (Financial Settlements & Bank Transactions)
120 |  -- ------------------------------------------------------------
121 |  CREATE TABLE payment (
122 |      id              INT AUTO_INCREMENT PRIMARY KEY,
123 |      invoice_id      INT NOT NULL UNIQUE,
124 |      company_id      INT NOT NULL,
125 |      total_amount    DECIMAL(10,2) NOT NULL,
126 |      amount_received DECIMAL(10,2) DEFAULT 0.00,
127 |      remaining_amount DECIMAL(10,2) DEFAULT 0.00,
128 |      payment_date    DATE,
129 |      payment_status  VARCHAR(20) DEFAULT 'Pending',
130 |      notes           VARCHAR(200),
131 |      FOREIGN KEY (invoice_id) REFERENCES invoice(id) ON DELETE CASCADE,
132 |      FOREIGN KEY (company_id) REFERENCES company(id) ON DELETE CASCADE
133 |  );
134 |
135 |  -- ============================================================
136 |  -- REALISTIC SIMULATED PRODUCTION SEED DATA
137 |  -- Real Indian Logistics Corridors, Enterprise Clients & Fleet
138 |  -- ============================================================
139 |
140 |  -- 1. SEED COMPANIES (Enterprise Logistics Clients & Operators)
```

FILE: src/com/transport/db/DatabaseConnection.java **[DATABASE] | 86 lines**

MySQL Singleton Connection & Dynamic Environment Config

```
 1 |  package com.transport.db;
 2 |
 3 |  import java.sql.Connection;
 4 |  import java.sql.DriverManager;
 5 |  import java.sql.SQLException;
 6 |
 7 |  /**
 8 |   * DatabaseConnection.java
 9 |   *
10 |   * This class handles the MySQL database connection.
11 |   * It uses the Singleton pattern so only ONE connection object exists.
12 |   *
13 |   * KEY CONCEPTS USED:
14 |   * - Static method (class-level method, no object needed)
15 |   * - JDBC (Java Database Connectivity)
16 |   * - Exception Handling (try-catch)
17 |   */
18 |  public class DatabaseConnection {
19 |
20 |      // --- Default Database settings ---
21 |      private static final String DEFAULT_HOST = "localhost";
22 |      private static final String DEFAULT_PORT = "3306";
23 |      private static final String DEFAULT_DB   = "transport_db";
24 |      private static final String DEFAULT_USER = "root";
25 |      private static final String DEFAULT_PASS = "";
26 |
27 |      // Private constructor - nobody can create an object of this class
28 |      private DatabaseConnection() {}
29 |
30 |      /**
31 |       * Constructs the JDBC URL dynamically from environment variables,
32 |       * falling back to default localhost settings.
33 |       */
34 |      public static String getJdbcUrl() {
35 |          String envUrl = System.getenv("DB_URL");
36 |          if (envUrl != null && !envUrl.trim().isEmpty()) {
37 |              return envUrl;
38 |          }
39 |          String host = System.getenv("DB_HOST");
40 |          if (host == null || host.trim().isEmpty()) host = DEFAULT_HOST;
41 |          String port = System.getenv("DB_PORT");
42 |          if (port == null || port.trim().isEmpty()) port = DEFAULT_PORT;
43 |          String db = System.getenv("DB_NAME");
44 |          if (db == null || db.trim().isEmpty()) db = DEFAULT_DB;
45 |
46 |          return "jdbc:mysql://" + host + ":" + port + "/" + db + "?allowPublicKeyRetrieval=tru...
47 |      }
48 |
49 |      public static String getJdbcUser() {
50 |          String user = System.getenv("DB_USER");
51 |          return (user != null && !user.trim().isEmpty()) ? user : DEFAULT_USER;
52 |      }
53 |
54 |      public static String getJdbcPassword() {
55 |          String pass = System.getenv("DB_PASSWORD");
56 |          return (pass != null) ? pass : DEFAULT_PASS;
57 |      }
58 |
59 |      /**
60 |       * Returns a new database connection every time it is called.
61 |       * Caller is responsible for closing the connection.
62 |       */
```

```
63 |      public static Connection getConnection() throws SQLException {
64 |          try {
65 |              // Load MySQL JDBC driver
66 |              Class.forName("com.mysql.cj.jdbc.Driver");
67 |          } catch (ClassNotFoundException e) {
68 |              System.out.println("MySQL Driver not found! Add mysql-connector.jar to lib folder...
69 |              throw new SQLException("Driver not found", e);
70 |          }
71 |          return DriverManager.getConnection(getJdbcUrl(), getJdbcUser(), getJdbcPassword());
72 |      }
73 |
74 |      /**
75 |       * Quick test to check if connection works.
76 |       * Run this once to verify your database is set up correctly.
77 |       */
78 |      public static boolean testConnection() {
79 |          try (Connection conn = getConnection()) {
80 |              return conn != null && !conn.isClosed();
81 |          } catch (SQLException e) {
82 |              System.out.println("Connection failed: " + e.getMessage());
83 |              return false;
84 |          }
85 |      }
86 |  }
```

FILE: src/com/transport/model/Trip.java **[MODEL] | 95 lines**

Trip Domain Model & Financial Transit Entity

```
1 |  package com.transport.model;
2 |
3 |  import java.util.Date;
4 |
5 |  /**
6 |   * Trip.java - Model class for Daily Operational Trip
7 |   *
8 |   * Represents a single transport trip logged by a driver.
9 |   * Includes company, driver, vehicle, route, cargo goods description, amount, and status.
10 |  */
11 |  public class Trip {
12 |
13 |      private int    id;
14 |      private String tripCode;           // e.g., TRIP001
15 |      private Date   tripDate;
16 |      private int    companyId;
17 |      private String companyName;        // Joined from company
18 |      private int    driverId;
19 |      private String driverName;         // Joined from driver
20 |      private int    vehicleId;
21 |      private String vehicleNumber;      // Joined from vehicle
22 |      private String sourceLocation;
23 |      private String destination;
24 |      private String goodsDescription;   // Cargo / Container / Material description
25 |      private double amount;
26 |      private String tripStatus;         // e.g., Completed, In Transit, Cancelled
27 |      private String remarks;
28 |
29 |      public Trip() {
30 |          this.tripStatus = "Completed";
31 |      }
32 |
33 |      public Trip(int id, String tripCode, Date tripDate,
34 |                  int companyId, String companyName,
35 |                  int driverId, String driverName,
36 |                  int vehicleId, String vehicleNumber,
37 |                  String sourceLocation, String destination,
38 |                  String goodsDescription, double amount,
39 |                  String tripStatus, String remarks) {
40 |          this.id              = id;
41 |          this.tripCode        = tripCode;
42 |          this.tripDate        = tripDate;
43 |          this.companyId       = companyId;
44 |          this.companyName     = companyName;
45 |          this.driverId        = driverId;
46 |          this.driverName      = driverName;
47 |          this.vehicleId       = vehicleId;
48 |          this.vehicleNumber   = vehicleNumber;
49 |          this.sourceLocation  = sourceLocation;
50 |          this.destination     = destination;
51 |          this.goodsDescription = goodsDescription;
52 |          this.amount          = amount;
53 |          this.tripStatus      = tripStatus != null ? tripStatus : "Completed";
54 |          this.remarks         = remarks;
55 |      }
56 |
57 |      // --- Getters ---
58 |      public int    getId()              { return id; }
59 |      public String getTripCode()        { return tripCode; }
60 |      public Date   getTripDate()        { return tripDate; }
```

```
61 |      public int    getCompanyId()        { return companyId; }
62 |      public String getCompanyName()      { return companyName; }
63 |      public int    getDriverId()         { return driverId; }
64 |      public String getDriverName()       { return driverName; }
65 |      public int    getVehicleId()        { return vehicleId; }
66 |      public String getVehicleNumber()    { return vehicleNumber; }
67 |      public String getSourceLocation()   { return sourceLocation; }
68 |      public String getDestination()      { return destination; }
69 |      public String getGoodsDescription() { return goodsDescription; }
70 |      public double getAmount()           { return amount; }
71 |      public String getTripStatus()       { return tripStatus; }
72 |      public String getRemarks()          { return remarks; }
73 |
74 |      // --- Setters ---
75 |      public void setId(int id)                              { this.id = id; }
76 |      public void setTripCode(String tripCode)               { this.tripCode = tripCode; }
77 |      public void setTripDate(Date tripDate)                 { this.tripDate = tripDate; }
78 |      public void setCompanyId(int companyId)                { this.companyId = companyId; }
79 |      public void setCompanyName(String name)                { this.companyName = name; }
80 |      public void setDriverId(int driverId)                  { this.driverId = driverId; }
81 |      public void setDriverName(String name)                 { this.driverName = name; }
82 |      public void setVehicleId(int vehicleId)                { this.vehicleId = vehicleId; }
83 |      public void setVehicleNumber(String number)            { this.vehicleNumber = number; }
84 |      public void setSourceLocation(String source)           { this.sourceLocation = source; }
85 |      public void setDestination(String dest)                { this.destination = dest; }
86 |      public void setGoodsDescription(String goodsDescription) { this.goodsDescription = goodsD...
87 |      public void setAmount(double amount)                   { this.amount = amount; }
88 |      public void setTripStatus(String tripStatus)           { this.tripStatus = tripStatus; }
89 |      public void setRemarks(String remarks)                 { this.remarks = remarks; }
90 |
91 |      @Override
92 |      public String toString() {
93 |          return tripCode + " | " + sourceLocation + " -> " + destination + " (INR " + amount +...
94 |      }
95 |  }
```

FILE: src/com/transport/model/Invoice.java **[MODEL] | 104 lines**

Monthly Billing & Transport Invoice Domain Model

```
1  |  package com.transport.model;
2  |
3  |  import java.util.Date;
4  |
5  |  /**
6  |   * Invoice.java - Model class for Monthly Transport Invoice
7  |   *
8  |   * Represents both Company-wide monthly invoices and Driver-wise monthly invoices.
9  |   */
10 |  public class Invoice {
11 |
12 |      private int    id;
13 |      private String invoiceNumber;      // e.g., INV-2026-001
14 |      private Date   invoiceDate;
15 |      private int    companyId;
16 |      private String companyName;        // Joined
17 |      private int    driverId;           // Optional (for driver-wise invoice)
18 |      private String driverName;         // Joined
19 |      private int    vehicleId;          // Optional
20 |      private String vehicleNumber;      // Joined
21 |      private int    billingMonth;       // 1 - 12
22 |      private int    billingYear;        // e.g. 2026
23 |      private int    totalTrips;
24 |      private double totalAmount;
25 |      private double amountPaid;
26 |      private double remainingAmount;
27 |      private String invoiceType;        // "Company" or "Driver"
28 |      private String status;             // "Generated", "Paid", "Partially Paid", "Pending"
29 |
30 |      public Invoice() {
31 |          this.invoiceType = "Company";
32 |          this.status      = "Generated";
33 |      }
34 |
35 |      public Invoice(int id, String invoiceNumber, Date invoiceDate,
36 |                     int companyId, String companyName,
37 |                     int driverId, String driverName,
38 |                     int vehicleId, String vehicleNumber,
39 |                     int billingMonth, int billingYear,
40 |                     int totalTrips, double totalAmount,
41 |                     double amountPaid, double remainingAmount,
42 |                     String invoiceType, String status) {
43 |          this.id              = id;
44 |          this.invoiceNumber   = invoiceNumber;
45 |          this.invoiceDate     = invoiceDate;
46 |          this.companyId       = companyId;
47 |          this.companyName     = companyName;
48 |          this.driverId        = driverId;
49 |          this.driverName      = driverName;
```

```
50 |          this.vehicleId       = vehicleId;
51 |          this.vehicleNumber   = vehicleNumber;
52 |          this.billingMonth    = billingMonth;
53 |          this.billingYear     = billingYear;
54 |          this.totalTrips      = totalTrips;
55 |          this.totalAmount     = totalAmount;
56 |          this.amountPaid      = amountPaid;
57 |          this.remainingAmount = remainingAmount;
58 |          this.invoiceType     = invoiceType != null ? invoiceType : "Company";
59 |          this.status          = status != null ? status : "Generated";
60 |      }
61 |
62 |      // --- Getters ---
63 |      public int    getId()              { return id; }
64 |      public String getInvoiceNumber()   { return invoiceNumber; }
65 |      public Date   getInvoiceDate()     { return invoiceDate; }
66 |      public int    getCompanyId()       { return companyId; }
67 |      public String getCompanyName()     { return companyName; }
68 |      public int    getDriverId()        { return driverId; }
69 |      public String getDriverName()      { return driverName; }
70 |      public int    getVehicleId()       { return vehicleId; }
71 |      public String getVehicleNumber()   { return vehicleNumber; }
72 |      public int    getBillingMonth()    { return billingMonth; }
73 |      public int    getBillingYear()     { return billingYear; }
74 |      public int    getTotalTrips()      { return totalTrips; }
75 |      public double getTotalAmount()     { return totalAmount; }
76 |      public double getAmountPaid()      { return amountPaid; }
77 |      public double getRemainingAmount() { return remainingAmount; }
78 |      public String getInvoiceType()     { return invoiceType; }
79 |      public String getStatus()          { return status; }
80 |
81 |      // --- Setters ---
82 |      public void setId(int id)                      { this.id = id; }
83 |      public void setInvoiceNumber(String num)       { this.invoiceNumber = num; }
84 |      public void setInvoiceDate(Date date)          { this.invoiceDate = date; }
85 |      public void setCompanyId(int id)               { this.companyId = id; }
86 |      public void setCompanyName(String name)        { this.companyName = name; }
87 |      public void setDriverId(int id)                { this.driverId = id; }
88 |      public void setDriverName(String name)         { this.driverName = name; }
89 |      public void setVehicleId(int id)               { this.vehicleId = id; }
90 |      public void setVehicleNumber(String num)       { this.vehicleNumber = num; }
91 |      public void setBillingMonth(int month)         { this.billingMonth = month; }
92 |      public void setBillingYear(int year)           { this.billingYear = year; }
93 |      public void setTotalTrips(int trips)           { this.totalTrips = trips; }
94 |      public void setTotalAmount(double amount)      { this.totalAmount = amount; }
95 |      public void setAmountPaid(double amountPaid)   { this.amountPaid = amountPaid; }
96 |      public void setRemainingAmount(double rem)     { this.remainingAmount = rem; }
97 |      public void setInvoiceType(String invoiceType) { this.invoiceType = invoiceType; }
98 |      public void setStatus(String status)           { this.status = status; }
99 |
100|      @Override
101|      public String toString() {
102|          return invoiceNumber + " | " + companyName + " (" + billingMonth + "/" + billingYear ...
103|      }
104|  }
```

FILE: src/com/transport/model/Driver.java **[MODEL] | 74 lines**

Driver & Vehicle Owner Identity Model

```
 1 |  package com.transport.model;
 2 |
 3 |  /**
 4 |   * Driver.java - Model class for Driver / Vehicle Owner
 5 |   *
 6 |   * Represents an individual driver or vehicle owner in MOT Logistics.
 7 |   * Drivers can own/operate multiple vehicles and work across multiple companies.
 8 |   */
 9 |  public class Driver {
10 |
11 |      private int     id;
12 |      private String  driverName;
13 |      private String  mobileNumber;
14 |      private String  email;
15 |      private String  licenceNumber;
16 |      private String  username;
17 |      private String  password;
18 |      private boolean isActive;
19 |      private int     companyId; // Optional link to registered/primary company
20 |
21 |      public Driver() {}
22 |
23 |      public Driver(int id, String driverName, String mobileNumber, String email,
24 |                    String licenceNumber, String username, String password, boolean isActive) {
25 |          this.id            = id;
26 |          this.driverName    = driverName;
27 |          this.mobileNumber  = mobileNumber;
28 |          this.email         = email;
29 |          this.licenceNumber = licenceNumber;
```

```
        this.username      = username;
        this.password      = password;
        this.isActive      = isActive;
    }

    public Driver(int id, String driverName, String mobileNumber, String email,
                  String licenceNumber, String username, String password, boolean isActive, i...
        this.id            = id;
        this.driverName    = driverName;
        this.mobileNumber  = mobileNumber;
        this.email         = email;
        this.licenceNumber = licenceNumber;
        this.username      = username;
        this.password      = password;
        this.isActive      = isActive;
        this.companyId     = companyId;
    }

    // --- Getters ---
    public int     getId()            { return id; }
    public String  getDriverName()    { return driverName; }
    public String  getMobileNumber()  { return mobileNumber; }
    public String  getEmail()         { return email; }
    public String  getLicenceNumber() { return licenceNumber; }
    public String  getUsername()      { return username; }
    public String  getPassword()      { return password; }
    public boolean isActive()         { return isActive; }
    public int     getCompanyId()     { return companyId; }

    // --- Setters ---
    public void setId(int id)                   { this.id = id; }
    public void setDriverName(String name)      { this.driverName = name; }
    public void setMobileNumber(String mobile)  { this.mobileNumber = mobile; }
    public void setEmail(String email)          { this.email = email; }
    public void setLicenceNumber(String lic)    { this.licenceNumber = lic; }
    public void setUsername(String username)    { this.username = username; }
    public void setPassword(String password)    { this.password = password; }
    public void setActive(boolean active)       { this.isActive = active; }
    public void setCompanyId(int companyId)     { this.companyId = companyId; }

    @Override
    public String toString() {
        return driverName;
    }
}
```

FILE: src/com/transport/model/Company.java **[MODEL] | 62 lines**

Enterprise Client Company Tenant Model

```
package com.transport.model;

/**
 * Company.java - Model class for Client Company
 *
 * Stores corporate client company profiles and their administrative credentials.
 */
public class Company {

    private int    id;
    private String companyName;
    private String adminName;
    private String email;
    private String username;
    private String password;
    private String contactNumber;
    private String address;
    private String gstNumber;

    public Company() {}

    public Company(int id, String companyName, String adminName, String email,
                   String username, String password, String contactNumber,
                   String address, String gstNumber) {
        this.id            = id;
        this.companyName   = companyName;
        this.adminName     = adminName;
        this.email         = email;
        this.username      = username;
        this.password      = password;
        this.contactNumber = contactNumber;
        this.address       = address;
        this.gstNumber     = gstNumber;
    }

    // --- Getters ---
    public int    getId()            { return id; }
    public String getCompanyName()   { return companyName; }
    public String getAdminName()     { return adminName; }
```

```
40 |      public String getEmail()         { return email; }
41 |      public String getUsername()      { return username; }
42 |      public String getPassword()      { return password; }
43 |      public String getContactNumber() { return contactNumber; }
44 |      public String getAddress()       { return address; }
45 |      public String getGstNumber()     { return gstNumber; }
46 |
47 |      // --- Setters ---
48 |      public void setId(int id)                   { this.id = id; }
49 |      public void setCompanyName(String name)     { this.companyName = name; }
50 |      public void setAdminName(String adminName)  { this.adminName = adminName; }
51 |      public void setEmail(String email)          { this.email = email; }
52 |      public void setUsername(String username)    { this.username = username; }
53 |      public void setPassword(String password)    { this.password = password; }
54 |      public void setContactNumber(String number) { this.contactNumber = number; }
55 |      public void setAddress(String address)      { this.address = address; }
56 |      public void setGstNumber(String gst)        { this.gstNumber = gst; }
57 |
58 |      @Override
59 |      public String toString() {
60 |          return companyName;
61 |      }
62 |  }
```

FILE: src/com/transport/model/Payment.java **[MODEL] | 72 lines**

Financial Settlement & Receivables Model

```
1  |  package com.transport.model;
2  |
3  |  import java.util.Date;
4  |
5  |  /**
6  |   * Payment.java - Model class for Invoice Payment
7  |   *
8  |   * Tracks billing settlements, received payments, remaining balances, and payment statuses.
9  |   */
10 |  public class Payment {
11 |
12 |      private int    id;
13 |      private int    invoiceId;
14 |      private int    companyId;
15 |      private String invoiceNumber;   // For display
16 |      private String companyName;     // For display
17 |      private double totalAmount;
18 |      private double amountReceived;
19 |      private double remainingAmount;
20 |      private Date   paymentDate;
21 |      private String paymentStatus;   // "Paid", "Partially Paid", "Pending"
22 |      private String notes;
23 |
24 |      public Payment() {}
25 |
26 |      public Payment(int id, int invoiceId, int companyId, String invoiceNumber,
27 |                     String companyName, double totalAmount, double amountReceived,
28 |                     double remainingAmount, Date paymentDate, String paymentStatus, String not...
29 |          this.id              = id;
30 |          this.invoiceId       = invoiceId;
31 |          this.companyId       = companyId;
32 |          this.invoiceNumber   = invoiceNumber;
33 |          this.companyName     = companyName;
34 |          this.totalAmount     = totalAmount;
35 |          this.amountReceived  = amountReceived;
36 |          this.remainingAmount = remainingAmount;
37 |          this.paymentDate     = paymentDate;
38 |          this.paymentStatus   = paymentStatus;
39 |          this.notes           = notes;
40 |      }
41 |
42 |      // --- Getters ---
43 |      public int    getId()              { return id; }
44 |      public int    getInvoiceId()       { return invoiceId; }
45 |      public int    getCompanyId()       { return companyId; }
46 |      public String getInvoiceNumber()   { return invoiceNumber; }
47 |      public String getCompanyName()     { return companyName; }
48 |      public double getTotalAmount()     { return totalAmount; }
49 |      public double getAmountReceived()  { return amountReceived; }
50 |      public double getRemainingAmount() { return remainingAmount; }
51 |      public Date   getPaymentDate()     { return paymentDate; }
52 |      public String getPaymentStatus()   { return paymentStatus; }
53 |      public String getNotes()           { return notes; }
54 |
55 |      // --- Setters ---
56 |      public void setId(int id)                      { this.id = id; }
57 |      public void setInvoiceId(int id)               { this.invoiceId = id; }
58 |      public void setCompanyId(int companyId)        { this.companyId = companyId; }
59 |      public void setInvoiceNumber(String num)       { this.invoiceNumber = num; }
60 |      public void setCompanyName(String name)        { this.companyName = name; }
61 |      public void setTotalAmount(double amt)         { this.totalAmount = amt; }
```

```
62 |      public void setAmountReceived(double amt)       { this.amountReceived = amt; }
63 |      public void setRemainingAmount(double amt)      { this.remainingAmount = amt; }
64 |      public void setPaymentDate(Date date)           { this.paymentDate = date; }
65 |      public void setPaymentStatus(String status)     { this.paymentStatus = status; }
66 |      public void setNotes(String notes)              { this.notes = notes; }
67 |
68 |      @Override
69 |      public String toString() {
70 |          return invoiceNumber + " | " + paymentStatus + " | INR " + totalAmount;
71 |      }
72 |  }
```

FILE: src/com/transport/model/Vehicle.java **[MODEL] | 53 lines**

Commercial Fleet Vehicle Asset Model

```
 1 |  package com.transport.model;
 2 |
 3 |  /**
 4 |   * Vehicle.java - Model class for Vehicle
 5 |   *
 6 |   * Represents a commercial vehicle (tempo, truck, etc.)
 7 |   * assigned to a driver.
 8 |   */
 9 |  public class Vehicle {
10 |
11 |      private int    id;
12 |      private String registrationNumber;
13 |      private String vehicleType;
14 |      private int    driverId;
15 |      private String driverName;  // For display purposes (joined from driver table)
16 |      private String status;
17 |
18 |      // Default constructor
19 |      public Vehicle() {}
20 |
21 |      // Parameterized constructor
22 |      public Vehicle(int id, String registrationNumber, String vehicleType,
23 |                     int driverId, String driverName, String status) {
24 |          this.id                = id;
25 |          this.registrationNumber = registrationNumber;
26 |          this.vehicleType        = vehicleType;
27 |          this.driverId           = driverId;
28 |          this.driverName         = driverName;
29 |          this.status             = status;
30 |      }
31 |
32 |      // --- Getters ---
33 |      public int    getId()                { return id; }
34 |      public String getRegistrationNumber() { return registrationNumber; }
35 |      public String getVehicleType()        { return vehicleType; }
36 |      public int    getDriverId()          { return driverId; }
37 |      public String getDriverName()        { return driverName; }
38 |      public String getStatus()            { return status; }
39 |
40 |      // --- Setters ---
41 |      public void setId(int id)                          { this.id = id; }
42 |      public void setRegistrationNumber(String regNo)    { this.registrationNumber = regNo; }
43 |      public void setVehicleType(String type)            { this.vehicleType = type; }
44 |      public void setDriverId(int driverId)              { this.driverId = driverId; }
45 |      public void setDriverName(String name)             { this.driverName = name; }
46 |      public void setStatus(String status)               { this.status = status; }
47 |
48 |      // Used by JComboBox to display vehicle number
49 |      @Override
50 |      public String toString() {
51 |          return registrationNumber;
52 |      }
53 |  }
```

FILE: src/com/transport/model/Admin.java **[MODEL] | 69 lines**

Company Admin Account & Security Model

```
 1 |  package com.transport.model;
 2 |
 3 |  /**
 4 |   * Admin.java - Model class for Company Administrator
 5 |   *
 6 |   * Represents a Company Admin session in MOT TRANS.
 7 |   * In this multi-tenant architecture, each company has its own independent admin.
 8 |   */
 9 |  public class Admin {
10 |
11 |      private int    id;             // Company ID
12 |      private int    companyId;      // Explicit alias for company ID
13 |      private String companyName;    // e.g., "Om Sai", "Oracle"
14 |      private String adminName;      // e.g., "Rajesh Patil"
15 |      private String email;          // Login email
```

```
16 |      private String username;        // Login username
17 |      private String password;
18 |      private String contactNumber;
19 |      private String address;
20 |      private String gstNumber;
21 |
22 |      public Admin() {}
23 |
24 |      public Admin(int id, String companyName, String adminName, String email,
25 |                   String username, String password, String contactNumber,
26 |                   String address, String gstNumber) {
27 |          this.id            = id;
28 |          this.companyId     = id;
29 |          this.companyName   = companyName;
30 |          this.adminName     = adminName;
31 |          this.email         = email;
32 |          this.username      = username;
33 |          this.password      = password;
34 |          this.contactNumber = contactNumber;
35 |          this.address       = address;
36 |          this.gstNumber     = gstNumber;
37 |      }
38 |
39 |      // --- Getters ---
40 |      public int    getId()            { return id; }
41 |      public int    getCompanyId()     { return companyId > 0 ? companyId : id; }
42 |      public String getCompanyName()   { return companyName; }
43 |      public String getAdminName()     { return adminName; }
44 |      public String getFullName()      { return adminName != null ? adminName : companyName; }
45 |      public String getEmail()         { return email; }
46 |      public String getUsername()      { return username; }
47 |      public String getPassword()      { return password; }
48 |      public String getContactNumber() { return contactNumber; }
49 |      public String getAddress()       { return address; }
50 |      public String getGstNumber()     { return gstNumber; }
51 |
52 |      // --- Setters ---
53 |      public void setId(int id)                      { this.id = id; this.companyId = id; }
54 |      public void setCompanyId(int companyId)        { this.companyId = companyId; this.id = c...
55 |      public void setCompanyName(String companyName) { this.companyName = companyName; }
56 |      public void setAdminName(String adminName)     { this.adminName = adminName; }
57 |      public void setFullName(String name)           { this.adminName = name; }
58 |      public void setEmail(String email)             { this.email = email; }
59 |      public void setUsername(String username)       { this.username = username; }
60 |      public void setPassword(String password)       { this.password = password; }
61 |      public void setContactNumber(String contact)   { this.contactNumber = contact; }
62 |      public void setAddress(String address)         { this.address = address; }
63 |      public void setGstNumber(String gstNumber)     { this.gstNumber = gstNumber; }
64 |
65 |      @Override
66 |      public String toString() {
67 |          return companyName + " Admin (" + adminName + ")";
68 |      }
69 |  }
```

FILE: src/com/transport/dao/TripDAO.java **[DAO] | 478 lines**

Trip Data Access, Tenant Isolation, Cross-Company Queries & Stats

```
1 |  package com.transport.dao;
2 |
3 |  import com.transport.db.DatabaseConnection;
4 |  import com.transport.model.Trip;
5 |  import java.sql.*;
6 |  import java.util.ArrayList;
7 |  import java.util.HashMap;
8 |  import java.util.List;
9 |  import java.util.Map;
10 |
11 |  /**
12 |   * TripDAO.java - Data Access Object for Trip Management
13 |   *
14 |   * Enforces strict multi-tenant isolation:
15 |   * - Company Admins can ONLY view trips for their own company_id.
16 |   * - Drivers can view their trips across multiple companies.
17 |   */
18 |  public class TripDAO {
19 |
20 |      /** Generate next sequential trip code like TRIP001, TRIP002, ... */
21 |      public String generateTripCode() {
22 |          String sql = "SELECT COUNT(*) FROM trip";
23 |          try (Connection conn = DatabaseConnection.getConnection();
24 |               PreparedStatement stmt = conn.prepareStatement(sql);
25 |               ResultSet rs = stmt.executeQuery()) {
26 |
27 |              if (rs.next()) {
28 |                  int count = rs.getInt(1) + 1;
29 |                  return String.format("TRIP%03d", count);
30 |              }
```

```
        } catch (SQLException e) {
            System.out.println("TripDAO.generateTripCode error: " + e.getMessage());
        }
        return "TRIP001";
    }

    /** Add a new trip entry */
    public boolean addTrip(Trip trip) {
        String sql = "INSERT INTO trip (trip_code, trip_date, company_id, driver_id, vehicle_...
                     "VALUES (?,?,?,?,?,?,?,?,?,?)";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, trip.getTripCode());
            stmt.setDate(2, new java.sql.Date(trip.getTripDate().getTime()));
            stmt.setInt(3, trip.getCompanyId());
            stmt.setInt(4, trip.getDriverId());
            stmt.setInt(5, trip.getVehicleId());
            stmt.setString(6, trip.getSourceLocation());
            stmt.setString(7, trip.getDestination());
            stmt.setString(8, trip.getGoodsDescription());
            stmt.setDouble(9, trip.getAmount());
            stmt.setString(10, trip.getTripStatus() != null ? trip.getTripStatus() : "Complet...
            stmt.setString(11, trip.getRemarks());

            return stmt.executeUpdate() > 0;

        } catch (SQLException e) {
            System.out.println("TripDAO.addTrip error: " + e.getMessage());
            return false;
        }
    }

    /** Update an existing trip record */
    public boolean updateTrip(Trip trip) {
        String sql = "UPDATE trip SET trip_date=?, company_id=?, driver_id=?, vehicle_id=?, s...

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setDate(1, new java.sql.Date(trip.getTripDate().getTime()));
            stmt.setInt(2, trip.getCompanyId());
            stmt.setInt(3, trip.getDriverId());
            stmt.setInt(4, trip.getVehicleId());
            stmt.setString(5, trip.getSourceLocation());
            stmt.setString(6, trip.getDestination());
            stmt.setString(7, trip.getGoodsDescription());
            stmt.setDouble(8, trip.getAmount());
            stmt.setString(9, trip.getTripStatus() != null ? trip.getTripStatus() : "Complete...
            stmt.setString(10, trip.getRemarks());
            stmt.setInt(11, trip.getId());

            return stmt.executeUpdate() > 0;

        } catch (SQLException e) {
            System.out.println("TripDAO.updateTrip error: " + e.getMessage());
            return false;
        }
    }

    /** Delete a trip by ID */
    public boolean deleteTrip(int id) {
        String sql = "DELETE FROM trip WHERE id=?";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, id);
            return stmt.executeUpdate() > 0;

        } catch (SQLException e) {
            System.out.println("TripDAO.deleteTrip error: " + e.getMessage());
            return false;
        }
    }

    /** Retrieve a single trip by ID */
    public Trip getTripById(int id) {
        String sql = "SELECT t.*, c.company_name, d.driver_name, v.registration_number " +
                     "FROM trip t " +
                     "LEFT JOIN company c ON t.company_id = c.id " +
                     "LEFT JOIN driver d ON t.driver_id = d.id " +
                     "LEFT JOIN vehicle v ON t.vehicle_id = v.id " +
                     "WHERE t.id=?";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
```

```
120 |
121 |              stmt.setInt(1, id);
122 |              ResultSet rs = stmt.executeQuery();
123 |              if (rs.next()) return mapRow(rs);
124 |
125 |          } catch (SQLException e) {
126 |              System.out.println("TripDAO.getTripById error: " + e.getMessage());
127 |          }
128 |          return null;
129 |      }
130 |
131 |      /**
132 |       * Flexible filter for trips by driver, company, vehicle, month, and year.
133 |       * Pass -1 to ignore any filter parameter.
134 |       */
135 |      public List<Trip> getTripsByFilter(int driverId, int companyId, int vehicleId, int month,...
136 |          return getTripsByFilter(driverId, companyId, vehicleId, month, year, null);
137 |      }
138 |
139 |      public List<Trip> getTripsByFilter(int driverId, int companyId, int vehicleId, int month,...
140 |          List<Trip> trips = new ArrayList<>();
141 |          StringBuilder sql = new StringBuilder(
142 |              "SELECT t.*, c.company_name, d.driver_name, v.registration_number " +
143 |              "FROM trip t " +
144 |              "JOIN company c ON t.company_id = c.id " +
145 |              "JOIN driver d ON t.driver_id = d.id " +
146 |              "JOIN vehicle v ON t.vehicle_id = v.id WHERE 1=1"
147 |          );
148 |
149 |          if (driverId > 0)  sql.append(" AND t.driver_id=?");
150 |          if (companyId > 0) sql.append(" AND t.company_id=?");
151 |          if (vehicleId > 0) sql.append(" AND t.vehicle_id=?");
152 |          if (month > 0)     sql.append(" AND MONTH(t.trip_date)=?");
153 |          if (year > 0)      sql.append(" AND YEAR(t.trip_date)=?");
154 |          if (searchKeyword != null && !searchKeyword.trim().isEmpty()) {
155 |              sql.append(" AND (LOWER(t.source_location) LIKE ? OR LOWER(t.destination) LIKE ? ...
156 |          }
157 |          sql.append(" ORDER BY t.trip_date DESC, t.id DESC");
158 |
159 |          try (Connection conn = DatabaseConnection.getConnection();
160 |               PreparedStatement stmt = conn.prepareStatement(sql.toString())) {
161 |
162 |              int idx = 1;
163 |              if (driverId > 0)  stmt.setInt(idx++, driverId);
164 |              if (companyId > 0) stmt.setInt(idx++, companyId);
165 |              if (vehicleId > 0) stmt.setInt(idx++, vehicleId);
166 |              if (month > 0)     stmt.setInt(idx++, month);
167 |              if (year > 0)      stmt.setInt(idx++, year);
168 |              if (searchKeyword != null && !searchKeyword.trim().isEmpty()) {
169 |                  String pattern = "%" + searchKeyword.trim().toLowerCase() + "%";
170 |                  stmt.setString(idx++, pattern);
171 |                  stmt.setString(idx++, pattern);
172 |                  stmt.setString(idx++, pattern);
173 |                  stmt.setString(idx++, pattern);
174 |                  stmt.setString(idx++, pattern);
175 |                  stmt.setString(idx++, pattern);
176 |                  stmt.setString(idx++, pattern);
177 |              }
178 |
179 |              ResultSet rs = stmt.executeQuery();
180 |              while (rs.next()) {
181 |                  trips.add(mapRow(rs));
182 |              }
183 |
184 |          } catch (SQLException e) {
185 |              System.out.println("TripDAO.getTripsByFilter error: " + e.getMessage());
186 |          }
187 |          return trips;
188 |      }
189 |
190 |      /** Total count of all trips entered in the system */
191 |      public int getTotalTripCount() {
192 |          String sql = "SELECT COUNT(*) FROM trip";
193 |          try (Connection conn = DatabaseConnection.getConnection();
194 |               PreparedStatement stmt = conn.prepareStatement(sql);
195 |               ResultSet rs = stmt.executeQuery()) {
196 |              if (rs.next()) return rs.getInt(1);
197 |          } catch (SQLException e) {
198 |              System.out.println("TripDAO.getTotalTripCount error: " + e.getMessage());
199 |          }
200 |          return 0;
201 |      }
202 |
203 |      /** Sum of earnings for the current month */
204 |      public double getCurrentMonthTotal() {
205 |          String sql = "SELECT COALESCE(SUM(amount), 0) FROM trip WHERE MONTH(trip_date)=MONTH(...
206 |          try (Connection conn = DatabaseConnection.getConnection();
207 |               PreparedStatement stmt = conn.prepareStatement(sql);
208 |               ResultSet rs = stmt.executeQuery()) {
```

```
209 |              if (rs.next()) return rs.getDouble(1);
210 |          } catch (SQLException e) {
211 |              System.out.println("TripDAO.getCurrentMonthTotal error: " + e.getMessage());
212 |          }
213 |          return 0.0;
214 |      }
215 |
216 |      /**
217 |       * Strictly isolated trips query for Company Admin.
218 |       * Guaranteed never to leak other companies' data.
219 |       */
220 |      public List<Trip> getTripsByCompany(int companyId) {
221 |          return getTripsByCompany(companyId, -1, -1, -1, -1, null);
222 |      }
223 |
224 |      public List<Trip> getTripsByCompany(int companyId, int driverId, int vehicleId, int month...
225 |          List<Trip> trips = new ArrayList<>();
226 |          StringBuilder sql = new StringBuilder(
227 |              "SELECT t.*, c.company_name, d.driver_name, v.registration_number " +
228 |              "FROM trip t " +
229 |              "JOIN company c ON t.company_id = c.id " +
230 |              "JOIN driver d ON t.driver_id = d.id " +
231 |              "JOIN vehicle v ON t.vehicle_id = v.id " +
232 |              "WHERE t.company_id = ?"
233 |          );
234 |
235 |          if (driverId > 0)  sql.append(" AND t.driver_id = ?");
236 |          if (vehicleId > 0) sql.append(" AND t.vehicle_id = ?");
237 |          if (month > 0)     sql.append(" AND MONTH(t.trip_date) = ?");
238 |          if (year > 0)      sql.append(" AND YEAR(t.trip_date) = ?");
239 |          if (searchKeyword != null && !searchKeyword.trim().isEmpty()) {
240 |              sql.append(" AND (LOWER(t.source_location) LIKE ? OR LOWER(t.destination) LIKE ? ...
241 |          }
242 |          sql.append(" ORDER BY t.trip_date DESC, t.id DESC");
243 |
244 |          try (Connection conn = DatabaseConnection.getConnection();
245 |               PreparedStatement stmt = conn.prepareStatement(sql.toString())) {
246 |
247 |              int idx = 1;
248 |              stmt.setInt(idx++, companyId);
249 |              if (driverId > 0)  stmt.setInt(idx++, driverId);
250 |              if (vehicleId > 0) stmt.setInt(idx++, vehicleId);
251 |              if (month > 0)     stmt.setInt(idx++, month);
252 |              if (year > 0)      stmt.setInt(idx++, year);
253 |              if (searchKeyword != null && !searchKeyword.trim().isEmpty()) {
254 |                  String pattern = "%" + searchKeyword.trim().toLowerCase() + "%";
255 |                  stmt.setString(idx++, pattern);
256 |                  stmt.setString(idx++, pattern);
257 |                  stmt.setString(idx++, pattern);
258 |                  stmt.setString(idx++, pattern);
259 |                  stmt.setString(idx++, pattern);
260 |                  stmt.setString(idx++, pattern);
261 |              }
262 |
263 |              ResultSet rs = stmt.executeQuery();
264 |              while (rs.next()) {
265 |                  trips.add(mapRow(rs));
266 |              }
267 |
268 |          } catch (SQLException e) {
269 |              System.out.println("TripDAO.getTripsByCompany error: " + e.getMessage());
270 |          }
271 |          return trips;
272 |      }
273 |
274 |      /**
275 |       * Retrieves trips for a driver across all companies (or filtered by company).
276 |       */
277 |      public List<Trip> getTripsByDriver(int driverId) {
278 |          return getTripsByDriver(driverId, -1, -1, -1, -1, null);
279 |      }
280 |
281 |      public List<Trip> getTripsByDriver(int driverId, int companyId, int vehicleId, int month,...
282 |          List<Trip> trips = new ArrayList<>();
283 |          StringBuilder sql = new StringBuilder(
284 |              "SELECT t.*, c.company_name, d.driver_name, v.registration_number " +
285 |              "FROM trip t " +
286 |              "JOIN company c ON t.company_id = c.id " +
287 |              "JOIN driver d ON t.driver_id = d.id " +
288 |              "JOIN vehicle v ON t.vehicle_id = v.id " +
289 |              "WHERE t.driver_id = ?"
290 |          );
291 |
292 |          if (companyId > 0) sql.append(" AND t.company_id = ?");
293 |          if (vehicleId > 0) sql.append(" AND t.vehicle_id = ?");
294 |          if (month > 0)     sql.append(" AND MONTH(t.trip_date) = ?");
295 |          if (year > 0)      sql.append(" AND YEAR(t.trip_date) = ?");
296 |          if (searchKeyword != null && !searchKeyword.trim().isEmpty()) {
297 |              sql.append(" AND (LOWER(t.source_location) LIKE ? OR LOWER(t.destination) LIKE ? ...
```

```
298 |          }
299 |          sql.append(" ORDER BY t.trip_date DESC, t.id DESC");
300 |
301 |          try (Connection conn = DatabaseConnection.getConnection();
302 |               PreparedStatement stmt = conn.prepareStatement(sql.toString())) {
303 |
304 |              int idx = 1;
305 |              stmt.setInt(idx++, driverId);
306 |              if (companyId > 0) stmt.setInt(idx++, companyId);
307 |              if (vehicleId > 0) stmt.setInt(idx++, vehicleId);
308 |              if (month > 0)     stmt.setInt(idx++, month);
309 |              if (year > 0)      stmt.setInt(idx++, year);
310 |              if (searchKeyword != null && !searchKeyword.trim().isEmpty()) {
311 |                  String pattern = "%" + searchKeyword.trim().toLowerCase() + "%";
312 |                  stmt.setString(idx++, pattern);
313 |                  stmt.setString(idx++, pattern);
314 |                  stmt.setString(idx++, pattern);
315 |                  stmt.setString(idx++, pattern);
316 |                  stmt.setString(idx++, pattern);
317 |                  stmt.setString(idx++, pattern);
318 |              }
319 |
320 |              ResultSet rs = stmt.executeQuery();
321 |              while (rs.next()) {
322 |                  trips.add(mapRow(rs));
323 |              }
324 |
325 |          } catch (SQLException e) {
326 |              System.out.println("TripDAO.getTripsByDriver error: " + e.getMessage());
327 |          }
328 |          return trips;
329 |      }
330 |
331 |      /** Retrieve trips for invoice generation */
332 |      public List<Trip> getTripsForInvoice(int companyId, int driverId, int vehicleId, int mont...
333 |          List<Trip> trips = new ArrayList<>();
334 |
335 |          StringBuilder sql = new StringBuilder(
336 |              "SELECT t.*, c.company_name, d.driver_name, v.registration_number " +
337 |              "FROM trip t " +
338 |              "JOIN company c ON t.company_id = c.id " +
339 |              "JOIN driver d ON t.driver_id = d.id " +
340 |              "JOIN vehicle v ON t.vehicle_id = v.id " +
341 |              "WHERE t.company_id = ?"
342 |          );
343 |
344 |          if (year > 0)      sql.append(" AND YEAR(t.trip_date) = ?");
345 |          if (month > 0)     sql.append(" AND MONTH(t.trip_date) = ?");
346 |          if (driverId > 0)  sql.append(" AND t.driver_id = ?");
347 |          if (vehicleId > 0) sql.append(" AND t.vehicle_id = ?");
348 |          sql.append(" ORDER BY t.trip_date ASC");
349 |
350 |          try (Connection conn = DatabaseConnection.getConnection();
351 |               PreparedStatement stmt = conn.prepareStatement(sql.toString())) {
352 |
353 |              int idx = 1;
354 |              stmt.setInt(idx++, companyId);
355 |              if (year > 0)      stmt.setInt(idx++, year);
356 |              if (month > 0)     stmt.setInt(idx++, month);
357 |              if (driverId > 0)  stmt.setInt(idx++, driverId);
358 |              if (vehicleId > 0) stmt.setInt(idx++, vehicleId);
359 |
360 |              ResultSet rs = stmt.executeQuery();
361 |              while (rs.next()) {
362 |                  trips.add(mapRow(rs));
363 |              }
364 |
365 |          } catch (SQLException e) {
366 |              System.out.println("TripDAO.getTripsForInvoice error: " + e.getMessage());
367 |          }
368 |          return trips;
369 |      }
370 |
371 |      /**
372 |       * Compute statistics strictly isolated to a specific Company.
373 |       */
374 |      public Map<String, Object> getCompanyStats(int companyId) {
375 |          Map<String, Object> stats = new HashMap<>();
376 |
377 |          String sql = "SELECT " +
378 |                       "  (SELECT COUNT(*) FROM trip WHERE company_id = ?) AS total_trips, " +
379 |                       "  (SELECT COUNT(DISTINCT driver_id) FROM trip WHERE company_id = ?) AS ...
380 |                       "  (SELECT COUNT(DISTINCT vehicle_id) FROM trip WHERE company_id = ?) AS...
381 |                       "  (SELECT COALESCE(SUM(amount), 0) FROM trip WHERE company_id = ? AND M...
382 |                       "  (SELECT COALESCE(SUM(amount_paid), 0) FROM invoice WHERE company_id =...
383 |                       "  (SELECT COALESCE(SUM(remaining_amount), 0) FROM invoice WHERE company...
384 |                       "  (SELECT COUNT(*) FROM invoice WHERE company_id = ? AND status = 'Paid...
385 |                       "  (SELECT COUNT(*) FROM invoice WHERE company_id = ? AND status != 'Pai...
386 |
```

```
387 |           try (Connection conn = DatabaseConnection.getConnection();
388 |                PreparedStatement stmt = conn.prepareStatement(sql)) {
389 |
390 |               for (int i = 1; i <= 8; i++) {
391 |                   stmt.setInt(i, companyId);
392 |               }
393 |
394 |               ResultSet rs = stmt.executeQuery();
395 |               if (rs.next()) {
396 |                   stats.put("totalTrips",           rs.getInt("total_trips"));
397 |                   stats.put("totalDrivers",         rs.getInt("total_drivers"));
398 |                   stats.put("totalVehicles",        rs.getInt("total_vehicles"));
399 |                   stats.put("currentMonthBilling",  rs.getDouble("current_month_billing"));
400 |                   stats.put("totalPaidAmount",      rs.getDouble("total_paid_amount"));
401 |                   stats.put("totalPendingAmount",   rs.getDouble("total_pending_amount"));
402 |                   stats.put("paidInvoicesCount",    rs.getInt("paid_invoices_count"));
403 |                   stats.put("pendingInvoicesCount", rs.getInt("pending_invoices_count"));
404 |               }
405 |
406 |           } catch (SQLException e) {
407 |               System.out.println("TripDAO.getCompanyStats error: " + e.getMessage());
408 |           }
409 |           return stats;
410 |       }
411 |
412 |       /**
413 |        * Compute statistics for a specific Driver.
414 |        */
415 |       public Map<String, Object> getDriverStats(int driverId) {
416 |           Map<String, Object> stats = new HashMap<>();
417 |
418 |           String sql = "SELECT " +
419 |                        "  (SELECT COUNT(*) FROM trip WHERE driver_id = ?) AS total_trips, " +
420 |                        "  (SELECT COUNT(DISTINCT company_id) FROM trip WHERE driver_id = ?) AS ...
421 |                        "  (SELECT COALESCE(SUM(amount), 0) FROM trip WHERE driver_id = ?) AS to...
422 |                        "  (SELECT COUNT(*) FROM trip WHERE driver_id = ? AND MONTH(trip_date) =...
423 |                        "  (SELECT COALESCE(SUM(amount_paid), 0) FROM invoice WHERE driver_id = ...
424 |                        "  (SELECT COALESCE(SUM(remaining_amount), 0) FROM invoice WHERE driver_...
425 |
426 |           try (Connection conn = DatabaseConnection.getConnection();
427 |                PreparedStatement stmt = conn.prepareStatement(sql)) {
428 |
429 |               for (int i = 1; i <= 6; i++) {
430 |                   stmt.setInt(i, driverId);
431 |               }
432 |
433 |               ResultSet rs = stmt.executeQuery();
434 |               if (rs.next()) {
435 |                   double totalEarnings = rs.getDouble("total_earnings");
436 |                   double paidAmount    = rs.getDouble("paid_amount");
437 |                   double pendingAmount = rs.getDouble("pending_amount");
438 |
439 |                   // If invoice-level pending is 0 but earnings exist without an invoice, fallb...
440 |                   if (pendingAmount == 0 && totalEarnings > paidAmount) {
441 |                       pendingAmount = totalEarnings - paidAmount;
442 |                   }
443 |
444 |                   stats.put("totalTrips",          rs.getInt("total_trips"));
445 |                   stats.put("totalCompanies",      rs.getInt("total_companies"));
446 |                   stats.put("totalEarnings",       totalEarnings);
447 |                   stats.put("currentMonthTrips",   rs.getInt("current_month_trips"));
448 |                   stats.put("paidAmount",          paidAmount);
449 |                   stats.put("pendingAmount",       pendingAmount);
450 |               }
451 |
452 |           } catch (SQLException e) {
453 |               System.out.println("TripDAO.getDriverStats error: " + e.getMessage());
454 |           }
455 |           return stats;
456 |       }
457 |
458 |       /** Helper: convert ResultSet row into a Trip object */
459 |       private Trip mapRow(ResultSet rs) throws SQLException {
460 |           Trip t = new Trip();
461 |           t.setId(rs.getInt("id"));
462 |           t.setTripCode(rs.getString("trip_code"));
463 |           t.setTripDate(rs.getDate("trip_date"));
464 |           t.setCompanyId(rs.getInt("company_id"));
465 |           t.setCompanyName(rs.getString("company_name"));
466 |           t.setDriverId(rs.getInt("driver_id"));
467 |           t.setDriverName(rs.getString("driver_name"));
468 |           t.setVehicleId(rs.getInt("vehicle_id"));
469 |           t.setVehicleNumber(rs.getString("registration_number"));
470 |           t.setSourceLocation(rs.getString("source_location"));
471 |           t.setDestination(rs.getString("destination"));
472 |           t.setGoodsDescription(rs.getString("goods_description"));
473 |           t.setAmount(rs.getDouble("amount"));
474 |           t.setTripStatus(rs.getString("trip_status"));
475 |           t.setRemarks(rs.getString("remarks"));
```

```
476 |          return t;
477 |      }
478 |  }
```

FILE: src/com/transport/dao/InvoiceDAO.java **[DAO] | 299 lines**

Automated Monthly Invoice Calculation, Ledger & Archival DAO

```
 1 |  package com.transport.dao;
 2 |
 3 |  import com.transport.db.DatabaseConnection;
 4 |  import com.transport.model.Invoice;
 5 |  import java.sql.*;
 6 |  import java.util.ArrayList;
 7 |  import java.util.List;
 8 |
 9 |  /**
10 |   * InvoiceDAO.java - Data Access Object for Invoice Operations
11 |   *
12 |   * Supports both Company-wide monthly invoices and Driver-wise monthly invoices.
13 |   * Enforces company-level and driver-level data scoping.
14 |   */
15 |  public class InvoiceDAO {
16 |
17 |      /** Generate next invoice number like INV-2026-001 */
18 |      public String generateInvoiceNumber() {
19 |          String sql = "SELECT COUNT(*) FROM invoice";
20 |          try (Connection conn = DatabaseConnection.getConnection();
21 |               PreparedStatement stmt = conn.prepareStatement(sql);
22 |               ResultSet rs = stmt.executeQuery()) {
23 |
24 |              if (rs.next()) {
25 |                  int count = rs.getInt(1) + 1;
26 |                  return String.format("INV-2026-%03d", count);
27 |              }
28 |
29 |          } catch (SQLException e) {
30 |              System.out.println("InvoiceDAO.generateInvoiceNumber error: " + e.getMessage());
31 |          }
32 |          return "INV-2026-001";
33 |      }
34 |
35 |      /** Save a new invoice record to database */
36 |      public boolean saveInvoice(Invoice invoice) {
37 |          String sql = "INSERT INTO invoice (invoice_number, invoice_date, company_id, driver_i...
38 |                       "VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?)";
39 |
40 |          try (Connection conn = DatabaseConnection.getConnection();
41 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
42 |
43 |              stmt.setString(1, invoice.getInvoiceNumber());
44 |              stmt.setDate(2, new java.sql.Date(invoice.getInvoiceDate().getTime()));
45 |              stmt.setInt(3, invoice.getCompanyId());
46 |
47 |              if (invoice.getDriverId() > 0) {
48 |                  stmt.setInt(4, invoice.getDriverId());
49 |              } else {
50 |                  stmt.setNull(4, Types.INTEGER);
51 |              }
52 |
53 |              if (invoice.getVehicleId() > 0) {
54 |                  stmt.setInt(5, invoice.getVehicleId());
55 |              } else {
56 |                  stmt.setNull(5, Types.INTEGER);
57 |              }
58 |
59 |              stmt.setInt(6, invoice.getBillingMonth());
60 |              stmt.setInt(7, invoice.getBillingYear());
61 |              stmt.setInt(8, invoice.getTotalTrips());
62 |              stmt.setDouble(9, invoice.getTotalAmount());
63 |              stmt.setDouble(10, invoice.getAmountPaid());
64 |              stmt.setDouble(11, invoice.getRemainingAmount());
65 |              stmt.setString(12, invoice.getInvoiceType() != null ? invoice.getInvoiceType() : ...
66 |              stmt.setString(13, invoice.getStatus() != null ? invoice.getStatus() : "Generated");
67 |
68 |              return stmt.executeUpdate() > 0;
69 |
70 |          } catch (SQLException e) {
71 |              System.out.println("InvoiceDAO.saveInvoice error: " + e.getMessage());
72 |              return false;
73 |          }
74 |      }
75 |
76 |      /** Record payment towards an invoice */
77 |      public boolean recordPayment(int invoiceId, int companyId, double paymentAmount, String n...
78 |          Invoice invoice = getInvoiceById(invoiceId);
79 |          if (invoice == null) return false;
80 |
81 |          double currentPaid = invoice.getAmountPaid();
```

```
82 |          double newPaid     = currentPaid + paymentAmount;
83 |          double total       = invoice.getTotalAmount();
84 |          double remaining   = Math.max(0.0, total - newPaid);
85 |
86 |          String newStatus;
87 |          if (remaining <= 0) {
88 |              newStatus = "Paid";
89 |          } else if (newPaid > 0) {
90 |              newStatus = "Partially Paid";
91 |          } else {
92 |              newStatus = "Pending";
93 |          }
94 |
95 |          String updateInvoiceSql = "UPDATE invoice SET amount_paid=?, remaining_amount=?, stat...
96 |          String upsertPaymentSql = "INSERT INTO payment (invoice_id, company_id, total_amount,...
97 |                                    "VALUES (?,?,?,?,?,?,?,?) " +
98 |                                    "ON DUPLICATE KEY UPDATE amount_received=?, remaining_amoun...
99 |
100|          Connection conn = null;
101|          try {
102|              conn = DatabaseConnection.getConnection();
103|              conn.setAutoCommit(false);
104|
105|              try (PreparedStatement invStmt = conn.prepareStatement(updateInvoiceSql);
106|                   PreparedStatement payStmt = conn.prepareStatement(upsertPaymentSql)) {
107|
108|                  invStmt.setDouble(1, newPaid);
109|                  invStmt.setDouble(2, remaining);
110|                  invStmt.setString(3, newStatus);
111|                  invStmt.setInt(4, invoiceId);
112|                  invStmt.executeUpdate();
113|
114|                  java.sql.Date pDate = paymentDate != null ? new java.sql.Date(paymentDate.get...
115|
116|                  payStmt.setInt(1, invoiceId);
117|                  payStmt.setInt(2, companyId);
118|                  payStmt.setDouble(3, total);
119|                  payStmt.setDouble(4, newPaid);
120|                  payStmt.setDouble(5, remaining);
121|                  payStmt.setDate(6, pDate);
122|                  payStmt.setString(7, newStatus);
123|                  payStmt.setString(8, notes);
124|
125|                  payStmt.setDouble(9, newPaid);
126|                  payStmt.setDouble(10, remaining);
127|                  payStmt.setDate(11, pDate);
128|                  payStmt.setString(12, newStatus);
129|                  payStmt.setString(13, notes);
130|
131|                  payStmt.executeUpdate();
132|
133|                  conn.commit();
134|                  return true;
135|              } catch (SQLException ex) {
136|                  conn.rollback();
137|                  throw ex;
138|              }
139|          } catch (SQLException e) {
140|              System.out.println("InvoiceDAO.recordPayment error: " + e.getMessage());
141|              return false;
142|          } finally {
143|              if (conn != null) {
144|                  try { conn.setAutoCommit(true); conn.close(); } catch (SQLException ignored) {}
145|              }
146|          }
147|      }
148|
149|      /** Retrieve all invoices across companies (general fallback) */
150|      public List<Invoice> getAllInvoices() {
151|          List<Invoice> invoices = new ArrayList<>();
152|          String sql = "SELECT i.*, c.company_name, d.driver_name, v.registration_number " +
153|                       "FROM invoice i " +
154|                       "JOIN company c ON i.company_id = c.id " +
155|                       "LEFT JOIN driver d ON i.driver_id = d.id " +
156|                       "LEFT JOIN vehicle v ON i.vehicle_id = v.id " +
157|                       "ORDER BY i.created_at DESC, i.id DESC";
158|
159|          try (Connection conn = DatabaseConnection.getConnection();
160|               PreparedStatement stmt = conn.prepareStatement(sql);
161|               ResultSet rs = stmt.executeQuery()) {
162|
163|              while (rs.next()) {
164|                  invoices.add(mapRow(rs));
165|              }
166|
167|          } catch (SQLException e) {
168|              System.out.println("InvoiceDAO.getAllInvoices error: " + e.getMessage());
169|          }
170|          return invoices;
```

```
171 |       }
172 |
173 |       /** Retrieve ID of last inserted invoice */
174 |       public int getLastInsertedInvoiceId() {
175 |           String sql = "SELECT MAX(id) FROM invoice";
176 |           try (Connection conn = DatabaseConnection.getConnection();
177 |                PreparedStatement stmt = conn.prepareStatement(sql);
178 |                ResultSet rs = stmt.executeQuery()) {
179 |
180 |               if (rs.next()) return rs.getInt(1);
181 |
182 |           } catch (SQLException e) {
183 |               System.out.println("InvoiceDAO.getLastInsertedInvoiceId error: " + e.getMessage());
184 |           }
185 |           return -1;
186 |       }
187 |
188 |       /** Total count of generated invoices */
189 |       public int getTotalInvoiceCount() {
190 |           String sql = "SELECT COUNT(*) FROM invoice";
191 |           try (Connection conn = DatabaseConnection.getConnection();
192 |                PreparedStatement stmt = conn.prepareStatement(sql);
193 |                ResultSet rs = stmt.executeQuery()) {
194 |
195 |               if (rs.next()) return rs.getInt(1);
196 |
197 |           } catch (SQLException e) {
198 |               System.out.println("InvoiceDAO.getTotalInvoiceCount error: " + e.getMessage());
199 |           }
200 |           return 0;
201 |       }
202 |
203 |       /** Retrieve invoices associated with a specific company (Company Admin) */
204 |       public List<Invoice> getInvoicesByCompany(int companyId) {
205 |           List<Invoice> invoices = new ArrayList<>();
206 |           String sql = "SELECT i.*, c.company_name, d.driver_name, v.registration_number " +
207 |                        "FROM invoice i " +
208 |                        "JOIN company c ON i.company_id = c.id " +
209 |                        "LEFT JOIN driver d ON i.driver_id = d.id " +
210 |                        "LEFT JOIN vehicle v ON i.vehicle_id = v.id " +
211 |                        "WHERE i.company_id = ? " +
212 |                        "ORDER BY i.created_at DESC, i.id DESC";
213 |
214 |           try (Connection conn = DatabaseConnection.getConnection();
215 |                PreparedStatement stmt = conn.prepareStatement(sql)) {
216 |
217 |               stmt.setInt(1, companyId);
218 |               ResultSet rs = stmt.executeQuery();
219 |               while (rs.next()) {
220 |                   invoices.add(mapRow(rs));
221 |               }
222 |
223 |           } catch (SQLException e) {
224 |               System.out.println("InvoiceDAO.getInvoicesByCompany error: " + e.getMessage());
225 |           }
226 |           return invoices;
227 |       }
228 |
229 |       /** Retrieve invoices associated with a specific driver (Driver Portal) */
230 |       public List<Invoice> getInvoicesByDriver(int driverId) {
231 |           List<Invoice> invoices = new ArrayList<>();
232 |           String sql = "SELECT i.*, c.company_name, d.driver_name, v.registration_number " +
233 |                        "FROM invoice i " +
234 |                        "JOIN company c ON i.company_id = c.id " +
235 |                        "LEFT JOIN driver d ON i.driver_id = d.id " +
236 |                        "LEFT JOIN vehicle v ON i.vehicle_id = v.id " +
237 |                        "WHERE i.driver_id = ? " +
238 |                        "ORDER BY i.created_at DESC, i.id DESC";
239 |
240 |           try (Connection conn = DatabaseConnection.getConnection();
241 |                PreparedStatement stmt = conn.prepareStatement(sql)) {
242 |
243 |               stmt.setInt(1, driverId);
244 |               ResultSet rs = stmt.executeQuery();
245 |               while (rs.next()) {
246 |                   invoices.add(mapRow(rs));
247 |               }
248 |
249 |           } catch (SQLException e) {
250 |               System.out.println("InvoiceDAO.getInvoicesByDriver error: " + e.getMessage());
251 |           }
252 |           return invoices;
253 |       }
254 |
255 |       /** Retrieve an invoice by ID */
256 |       public Invoice getInvoiceById(int id) {
257 |           String sql = "SELECT i.*, c.company_name, d.driver_name, v.registration_number " +
258 |                        "FROM invoice i " +
259 |                        "JOIN company c ON i.company_id = c.id " +
```

```
                        "LEFT JOIN driver d ON i.driver_id = d.id " +
                        "LEFT JOIN vehicle v ON i.vehicle_id = v.id " +
                        "WHERE i.id=?";

            try (Connection conn = DatabaseConnection.getConnection();
                 PreparedStatement stmt = conn.prepareStatement(sql)) {

                stmt.setInt(1, id);
                ResultSet rs = stmt.executeQuery();
                if (rs.next()) return mapRow(rs);

            } catch (SQLException e) {
                System.out.println("InvoiceDAO.getInvoiceById error: " + e.getMessage());
            }
            return null;
        }

        /** Helper: convert ResultSet row into an Invoice object */
        private Invoice mapRow(ResultSet rs) throws SQLException {
            Invoice inv = new Invoice();
            inv.setId(rs.getInt("id"));
            inv.setInvoiceNumber(rs.getString("invoice_number"));
            inv.setInvoiceDate(rs.getDate("invoice_date"));
            inv.setCompanyId(rs.getInt("company_id"));
            inv.setCompanyName(rs.getString("company_name"));
            inv.setDriverId(rs.getInt("driver_id"));
            inv.setDriverName(rs.getString("driver_name"));
            inv.setVehicleId(rs.getInt("vehicle_id"));
            inv.setVehicleNumber(rs.getString("registration_number"));
            inv.setBillingMonth(rs.getInt("billing_month"));
            inv.setBillingYear(rs.getInt("billing_year"));
            inv.setTotalTrips(rs.getInt("total_trips"));
            inv.setTotalAmount(rs.getDouble("total_amount"));
            inv.setAmountPaid(rs.getDouble("amount_paid"));
            inv.setRemainingAmount(rs.getDouble("remaining_amount"));
            inv.setInvoiceType(rs.getString("invoice_type"));
            inv.setStatus(rs.getString("status"));
            return inv;
        }
    }
```

FILE: src/com/transport/dao/DriverDAO.java **[DAO] | 408 lines**

Driver Authentication, Multi-Vehicle Operations & Company Roster

```
package com.transport.dao;

import com.transport.db.DatabaseConnection;
import com.transport.model.Driver;
import com.transport.model.DriverCompanySummary;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * DriverDAO.java - Data Access Object for Driver
 *
 * Handles database operations for Driver accounts:
 * - Driver authentication (Username or Email)
 * - Driver registration & profile updates
 * - Company-specific driver roster queries and driver onboarding by company
 * - Vehicle assignments and deactivation
 */
public class DriverDAO {

    /**
     * Authenticate driver login using username OR email and password.
     * Only active accounts are permitted.
     */
    public Driver login(String usernameOrEmail, String password) {
        String sql = "SELECT * FROM driver WHERE (username=? OR email=?) AND password=? AND i...

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, usernameOrEmail);
            stmt.setString(2, usernameOrEmail);
            stmt.setString(3, password);
            ResultSet rs = stmt.executeQuery();

            if (rs.next()) {
                return mapRow(rs);
            }

        } catch (SQLException e) {
            System.out.println("DriverDAO.login error: " + e.getMessage());
        }
        return null;
    }
```

```
    /** Add a new driver to the database */
    public boolean addDriver(Driver driver) {
        String sql = "INSERT INTO driver (driver_name, mobile_number, email, licence_number, ...
                     "VALUES (?,?,?,?,?,?,1,?)";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, driver.getDriverName());
            stmt.setString(2, driver.getMobileNumber());
            stmt.setString(3, driver.getEmail());
            stmt.setString(4, driver.getLicenceNumber());
            stmt.setString(5, driver.getUsername());
            stmt.setString(6, driver.getPassword());
            if (driver.getCompanyId() > 0) {
                stmt.setInt(7, driver.getCompanyId());
            } else {
                stmt.setNull(7, Types.INTEGER);
            }

            return stmt.executeUpdate() > 0;

        } catch (SQLException e) {
            System.out.println("DriverDAO.addDriver error: " + e.getMessage());
            return false;
        }
    }

    /**
     * Register a new driver along with their primary vehicle in an atomic transaction.
     */
    public boolean registerDriverWithVehicle(Driver driver, String vehicleReg, String vehicle...
        return registerDriverWithVehicle(driver, vehicleReg, vehicleType, driver.getCompanyId...
    }

    /**
     * Register a new driver along with their primary vehicle linked to a specific company.
     */
    public boolean registerDriverWithVehicle(Driver driver, String vehicleReg, String vehicle...
        String driverSql  = "INSERT INTO driver (driver_name, mobile_number, email, licence_n...
                            "VALUES (?,?,?,?,?,?,1,?)";
        String vehicleSql = "INSERT INTO vehicle (registration_number, vehicle_type, driver_i...

        Connection conn = null;
        try {
            conn = DatabaseConnection.getConnection();
            conn.setAutoCommit(false);

            int driverId = -1;
            try (PreparedStatement stmt = conn.prepareStatement(driverSql, Statement.RETURN_G...
                stmt.setString(1, driver.getDriverName());
                stmt.setString(2, driver.getMobileNumber());
                stmt.setString(3, driver.getEmail());
                stmt.setString(4, driver.getLicenceNumber());
                stmt.setString(5, driver.getUsername());
                stmt.setString(6, driver.getPassword());
                if (companyId > 0) {
                    stmt.setInt(7, companyId);
                } else {
                    stmt.setNull(7, Types.INTEGER);
                }
                stmt.executeUpdate();

                ResultSet rs = stmt.getGeneratedKeys();
                if (rs.next()) {
                    driverId = rs.getInt(1);
                }
            }

            if (driverId > 0 && vehicleReg != null && !vehicleReg.trim().isEmpty()) {
                try (PreparedStatement stmt = conn.prepareStatement(vehicleSql)) {
                    stmt.setString(1, vehicleReg.trim().toUpperCase());
                    stmt.setString(2, vehicleType != null ? vehicleType : "Truck");
                    stmt.setInt(3, driverId);
                    stmt.executeUpdate();
                }
            }

            conn.commit();
            return true;

        } catch (SQLException e) {
            System.out.println("DriverDAO.registerDriverWithVehicle error: " + e.getMessage());
            if (conn != null) {
                try { conn.rollback(); } catch (SQLException ex) { ex.printStackTrace(); }
            }
            return false;
        } finally {
```

```
134 |              if (conn != null) {
135 |                  try { conn.setAutoCommit(true); conn.close(); } catch (SQLException ex) { ex....
136 |              }
137 |          }
138 |      }
139 |
140 |      /**
141 |       * Add a driver directly from Company Admin panel.
142 |       */
143 |      public boolean addDriverForCompany(Driver driver, String vehicleReg, String vehicleType, ...
144 |          driver.setCompanyId(companyId);
145 |          return registerDriverWithVehicle(driver, vehicleReg, vehicleType, companyId);
146 |      }
147 |
148 |      /**
149 |       * Assign or link an existing driver to a company.
150 |       */
151 |      public boolean assignDriverToCompany(int driverId, int companyId) {
152 |          String sql = "UPDATE driver SET company_id = ? WHERE id = ?";
153 |
154 |          try (Connection conn = DatabaseConnection.getConnection();
155 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
156 |
157 |              stmt.setInt(1, companyId);
158 |              stmt.setInt(2, driverId);
159 |              return stmt.executeUpdate() > 0;
160 |
161 |          } catch (SQLException e) {
162 |              System.out.println("DriverDAO.assignDriverToCompany error: " + e.getMessage());
163 |              return false;
164 |          }
165 |      }
166 |
167 |      /** Update driver information */
168 |      public boolean updateDriver(Driver driver) {
169 |          String sql = "UPDATE driver SET driver_name=?, mobile_number=?, email=?, licence_numb...
170 |
171 |          try (Connection conn = DatabaseConnection.getConnection();
172 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
173 |
174 |              stmt.setString(1, driver.getDriverName());
175 |              stmt.setString(2, driver.getMobileNumber());
176 |              stmt.setString(3, driver.getEmail());
177 |              stmt.setString(4, driver.getLicenceNumber());
178 |              stmt.setString(5, driver.getUsername());
179 |              stmt.setString(6, driver.getPassword());
180 |              stmt.setInt(7, driver.getId());
181 |
182 |              return stmt.executeUpdate() > 0;
183 |
184 |          } catch (SQLException e) {
185 |              System.out.println("DriverDAO.updateDriver error: " + e.getMessage());
186 |              return false;
187 |          }
188 |      }
189 |
190 |      /** Update driver self-profile */
191 |      public boolean updateProfile(int driverId, String mobile, String email, String password) {
192 |          String sql;
193 |          boolean hasPassword = password != null && !password.trim().isEmpty();
194 |
195 |          if (hasPassword) {
196 |              sql = "UPDATE driver SET mobile_number=?, email=?, password=? WHERE id=?";
197 |          } else {
198 |              sql = "UPDATE driver SET mobile_number=?, email=? WHERE id=?";
199 |          }
200 |
201 |          try (Connection conn = DatabaseConnection.getConnection();
202 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
203 |
204 |              stmt.setString(1, mobile);
205 |              stmt.setString(2, email);
206 |              if (hasPassword) {
207 |                  stmt.setString(3, password);
208 |                  stmt.setInt(4, driverId);
209 |              } else {
210 |                  stmt.setInt(3, driverId);
211 |              }
212 |
213 |              return stmt.executeUpdate() > 0;
214 |
215 |          } catch (SQLException e) {
216 |              System.out.println("DriverDAO.updateProfile error: " + e.getMessage());
217 |              return false;
218 |          }
219 |      }
220 |
221 |      /** Reset password using username or email */
222 |      public boolean resetPassword(String usernameOrEmail, String newPassword) {
```

```
223 |           String sql = "UPDATE driver SET password=? WHERE username=? OR email=?";
224 |
225 |           try (Connection conn = DatabaseConnection.getConnection();
226 |                PreparedStatement stmt = conn.prepareStatement(sql)) {
227 |
228 |               stmt.setString(1, newPassword);
229 |               stmt.setString(2, usernameOrEmail);
230 |               stmt.setString(3, usernameOrEmail);
231 |
232 |               return stmt.executeUpdate() > 0;
233 |
234 |           } catch (SQLException e) {
235 |               System.out.println("DriverDAO.resetPassword error: " + e.getMessage());
236 |               return false;
237 |           }
238 |       }
239 |
240 |       /** Activate or deactivate a driver account */
241 |       public boolean setDriverActive(int driverId, boolean active) {
242 |           String sql = "UPDATE driver SET is_active=? WHERE id=?";
243 |
244 |           try (Connection conn = DatabaseConnection.getConnection();
245 |                PreparedStatement stmt = conn.prepareStatement(sql)) {
246 |
247 |               stmt.setInt(1, active ? 1 : 0);
248 |               stmt.setInt(2, driverId);
249 |               return stmt.executeUpdate() > 0;
250 |
251 |           } catch (SQLException e) {
252 |               System.out.println("DriverDAO.setDriverActive error: " + e.getMessage());
253 |               return false;
254 |           }
255 |       }
256 |
257 |       /** Retrieve all drivers (both active and inactive) */
258 |       public List<Driver> getAllDrivers() {
259 |           List<Driver> drivers = new ArrayList<>();
260 |           String sql = "SELECT * FROM driver ORDER BY driver_name";
261 |
262 |           try (Connection conn = DatabaseConnection.getConnection();
263 |                PreparedStatement stmt = conn.prepareStatement(sql);
264 |                ResultSet rs = stmt.executeQuery()) {
265 |
266 |               while (rs.next()) {
267 |                   drivers.add(mapRow(rs));
268 |               }
269 |
270 |           } catch (SQLException e) {
271 |               System.out.println("DriverDAO.getAllDrivers error: " + e.getMessage());
272 |           }
273 |           return drivers;
274 |       }
275 |
276 |       /** Retrieve only active drivers for assignment in dropdowns */
277 |       public List<Driver> getActiveDrivers() {
278 |           List<Driver> drivers = new ArrayList<>();
279 |           String sql = "SELECT * FROM driver WHERE is_active=1 ORDER BY driver_name";
280 |
281 |           try (Connection conn = DatabaseConnection.getConnection();
282 |                PreparedStatement stmt = conn.prepareStatement(sql);
283 |                ResultSet rs = stmt.executeQuery()) {
284 |
285 |               while (rs.next()) {
286 |                   drivers.add(mapRow(rs));
287 |               }
288 |
289 |           } catch (SQLException e) {
290 |               System.out.println("DriverDAO.getActiveDrivers error: " + e.getMessage());
291 |           }
292 |           return drivers;
293 |       }
294 |
295 |       /** Retrieve a driver by their primary key ID */
296 |       public Driver getDriverById(int id) {
297 |           String sql = "SELECT * FROM driver WHERE id=?";
298 |
299 |           try (Connection conn = DatabaseConnection.getConnection();
300 |                PreparedStatement stmt = conn.prepareStatement(sql)) {
301 |
302 |               stmt.setInt(1, id);
303 |               ResultSet rs = stmt.executeQuery();
304 |               if (rs.next()) return mapRow(rs);
305 |
306 |           } catch (SQLException e) {
307 |               System.out.println("DriverDAO.getDriverById error: " + e.getMessage());
308 |           }
309 |           return null;
310 |       }
311 |
```

```
312 |       /**
313 |        * Retrieves summary of drivers associated with a specific company:
314 |        * - Drivers who have completed trips for this company, OR
315 |        * - Drivers registered directly by this company.
316 |        * Returned columns: Driver Name, Vehicle Number, Total Trips.
317 |        */
318 |       public List<DriverCompanySummary> getDriversForCompany(int companyId) {
319 |           List<DriverCompanySummary> list = new ArrayList<>();
320 |           String sql = "SELECT d.id AS driver_id, d.driver_name, d.mobile_number, d.email, " +
321 |                        "       COALESCE(GROUP_CONCAT(DISTINCT v.registration_number ORDER BY v....
322 |                        "       COUNT(DISTINCT t.id) AS total_trips " +
323 |                        "FROM driver d " +
324 |                        "LEFT JOIN vehicle v ON v.driver_id = d.id " +
325 |                        "LEFT JOIN trip t ON t.driver_id = d.id AND t.company_id = ? " +
326 |                        "WHERE d.company_id = ? OR t.company_id = ? " +
327 |                        "GROUP BY d.id, d.driver_name, d.mobile_number, d.email " +
328 |                        "ORDER BY total_trips DESC, d.driver_name ASC";
329 |
330 |           try (Connection conn = DatabaseConnection.getConnection();
331 |                PreparedStatement stmt = conn.prepareStatement(sql)) {
332 |
333 |               stmt.setInt(1, companyId);
334 |               stmt.setInt(2, companyId);
335 |               stmt.setInt(3, companyId);
336 |               ResultSet rs = stmt.executeQuery();
337 |
338 |               while (rs.next()) {
339 |                   DriverCompanySummary s = new DriverCompanySummary();
340 |                   s.setDriverId(rs.getInt("driver_id"));
341 |                   s.setDriverName(rs.getString("driver_name"));
342 |                   s.setMobileNumber(rs.getString("mobile_number"));
343 |                   s.setEmail(rs.getString("email"));
344 |                   s.setVehicleNumbers(rs.getString("vehicle_numbers"));
345 |                   s.setTotalTrips(rs.getInt("total_trips"));
346 |                   list.add(s);
347 |               }
348 |
349 |           } catch (SQLException e) {
350 |               System.out.println("DriverDAO.getDriversForCompany error: " + e.getMessage());
351 |           }
352 |           return list;
353 |       }
354 |
355 |       /**
356 |        * Delete a driver by ID safely.
357 |        */
358 |       public boolean deleteDriver(int id) {
359 |           String unassignSql = "UPDATE vehicle SET driver_id = NULL WHERE driver_id = ?";
360 |           String deleteSql   = "DELETE FROM driver WHERE id = ?";
361 |
362 |           Connection conn = null;
363 |           try {
364 |               conn = DatabaseConnection.getConnection();
365 |               conn.setAutoCommit(false);
366 |
367 |               try (PreparedStatement unassignStmt = conn.prepareStatement(unassignSql);
368 |                    PreparedStatement deleteStmt   = conn.prepareStatement(deleteSql)) {
369 |
370 |                   unassignStmt.setInt(1, id);
371 |                   unassignStmt.executeUpdate();
372 |
373 |                   deleteStmt.setInt(1, id);
374 |                   int rows = deleteStmt.executeUpdate();
375 |
376 |                   conn.commit();
377 |                   return rows > 0;
378 |               } catch (SQLException ex) {
379 |                   conn.rollback();
380 |                   throw ex;
381 |               }
382 |           } catch (SQLException e) {
383 |               System.out.println("DriverDAO.deleteDriver error: " + e.getMessage());
384 |               return false;
385 |           } finally {
386 |               if (conn != null) {
387 |                   try { conn.setAutoCommit(true); conn.close(); } catch (SQLException ignored) {}
388 |               }
389 |           }
390 |       }
391 |
392 |       /** Helper: convert a ResultSet row into a Driver object */
393 |       private Driver mapRow(ResultSet rs) throws SQLException {
394 |           Driver d = new Driver();
395 |           d.setId(rs.getInt("id"));
396 |           d.setDriverName(rs.getString("driver_name"));
397 |           d.setMobileNumber(rs.getString("mobile_number"));
398 |           d.setEmail(rs.getString("email"));
399 |           d.setLicenceNumber(rs.getString("licence_number"));
400 |           d.setUsername(rs.getString("username"));
```

```
401 |          d.setPassword(rs.getString("password"));
402 |          d.setActive(rs.getInt("is_active") == 1);
403 |          try {
404 |              d.setCompanyId(rs.getInt("company_id"));
405 |          } catch (SQLException ignored) {}
406 |          return d;
407 |      }
408 |  }
```

FILE: src/com/transport/dao/CompanyDAO.java **[DAO] | 145 lines**

Company Tenant Auth, Security & Financial Dashboards DAO

```
  1 |  package com.transport.dao;
  2 |
  3 |  import com.transport.db.DatabaseConnection;
  4 |  import com.transport.model.Company;
  5 |  import java.sql.*;
  6 |  import java.util.ArrayList;
  7 |  import java.util.List;
  8 |
  9 |  /**
 10 |   * CompanyDAO.java - Data Access Object for Company
 11 |   *
 12 |   * Handles all database operations for the Company table:
 13 |   * - Add a new company
 14 |   * - Update company details
 15 |   * - Get all companies
 16 |   * - Get company by ID
 17 |   */
 18 |  public class CompanyDAO {
 19 |
 20 |      /** Add a new company to the database */
 21 |      public boolean addCompany(Company company) {
 22 |          String adminName = (company.getAdminName() != null && !company.getAdminName().trim()....
 23 |                  ? company.getAdminName() : company.getCompanyName() + " Admin";
 24 |          String email = (company.getEmail() != null && !company.getEmail().trim().isEmpty())
 25 |                  ? company.getEmail() : "contact_" + (System.currentTimeMillis() % 1000000) + ...
 26 |          String username = (company.getUsername() != null && !company.getUsername().trim().isE...
 27 |                  ? company.getUsername() : "comp_" + (System.currentTimeMillis() % 1000000);
 28 |          String password = (company.getPassword() != null && !company.getPassword().trim().isE...
 29 |                  ? company.getPassword() : "comp123";
 30 |
 31 |          String sql = "INSERT INTO company (company_name, admin_name, email, username, passwor...
 32 |                       "VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
 33 |
 34 |          try (Connection conn = DatabaseConnection.getConnection();
 35 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
 36 |
 37 |              stmt.setString(1, company.getCompanyName());
 38 |              stmt.setString(2, adminName);
 39 |              stmt.setString(3, email);
 40 |              stmt.setString(4, username);
 41 |              stmt.setString(5, password);
 42 |              stmt.setString(6, company.getAddress());
 43 |              stmt.setString(7, company.getContactNumber());
 44 |              stmt.setString(8, company.getGstNumber());
 45 |
 46 |              int rows = stmt.executeUpdate();
 47 |              return rows > 0; // true if insert was successful
 48 |
 49 |          } catch (SQLException e) {
 50 |              System.out.println("CompanyDAO.addCompany error: " + e.getMessage());
 51 |              return false;
 52 |          }
 53 |      }
 54 |
 55 |      /** Update an existing company */
 56 |      public boolean updateCompany(Company company) {
 57 |          String sql = "UPDATE company SET company_name=?, address=?, contact_number=?, gst_num...
 58 |
 59 |          try (Connection conn = DatabaseConnection.getConnection();
 60 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
 61 |
 62 |              stmt.setString(1, company.getCompanyName());
 63 |              stmt.setString(2, company.getAddress());
 64 |              stmt.setString(3, company.getContactNumber());
 65 |              stmt.setString(4, company.getGstNumber());
 66 |              stmt.setInt(5, company.getId());
 67 |
 68 |              int rows = stmt.executeUpdate();
 69 |              return rows > 0;
 70 |
 71 |          } catch (SQLException e) {
 72 |              System.out.println("CompanyDAO.updateCompany error: " + e.getMessage());
 73 |              return false;
 74 |          }
 75 |      }
 76 |
```

```
    /** Get all companies from database */
    public List<Company> getAllCompanies() {
        List<Company> companies = new ArrayList<>();
        String sql = "SELECT * FROM company ORDER BY company_name";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql);
             ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                Company c = new Company();
                c.setId(rs.getInt("id"));
                c.setCompanyName(rs.getString("company_name"));
                c.setAddress(rs.getString("address"));
                c.setContactNumber(rs.getString("contact_number"));
                c.setGstNumber(rs.getString("gst_number"));
                companies.add(c);
            }

        } catch (SQLException e) {
            System.out.println("CompanyDAO.getAllCompanies error: " + e.getMessage());
        }

        return companies;
    }

    /** Get one company by ID */
    public Company getCompanyById(int id) {
        String sql = "SELECT * FROM company WHERE id = ?";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, id);
            ResultSet rs = stmt.executeQuery();

            if (rs.next()) {
                Company c = new Company();
                c.setId(rs.getInt("id"));
                c.setCompanyName(rs.getString("company_name"));
                c.setAddress(rs.getString("address"));
                c.setContactNumber(rs.getString("contact_number"));
                c.setGstNumber(rs.getString("gst_number"));
                return c;
            }

        } catch (SQLException e) {
            System.out.println("CompanyDAO.getCompanyById error: " + e.getMessage());
        }

        return null;
    }

    /** Delete company by ID */
    public boolean deleteCompany(int id) {
        String sql = "DELETE FROM company WHERE id = ?";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, id);
            return stmt.executeUpdate() > 0;

        } catch (SQLException e) {
            System.out.println("CompanyDAO.deleteCompany error: " + e.getMessage());
            return false;
        }
    }
}
```

FILE: src/com/transport/dao/PaymentDAO.java

[DAO] | 179 lines

Payment Processing, Partial Settlements & Balance Tracking DAO

```
package com.transport.dao;

import com.transport.db.DatabaseConnection;
import com.transport.model.Payment;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * PaymentDAO.java - Data Access Object for Payment Tracking
 *
 * Implements company-level payment history and settlements.
 */
public class PaymentDAO {

```

```
16 |      /** Create a new payment record linked to an invoice */
17 |      public boolean createPayment(Payment payment) {
18 |          String sql = "INSERT INTO payment (invoice_id, company_id, total_amount, amount_recei...
19 |                       "VALUES (?,?,?,?,?,?,?)";
20 |
21 |          try (Connection conn = DatabaseConnection.getConnection();
22 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
23 |
24 |              stmt.setInt(1, payment.getInvoiceId());
25 |              stmt.setInt(2, payment.getCompanyId());
26 |              stmt.setDouble(3, payment.getTotalAmount());
27 |              stmt.setDouble(4, payment.getAmountReceived());
28 |              stmt.setDouble(5, payment.getRemainingAmount());
29 |              stmt.setString(6, payment.getPaymentStatus());
30 |              stmt.setString(7, payment.getNotes());
31 |
32 |              return stmt.executeUpdate() > 0;
33 |
34 |          } catch (SQLException e) {
35 |              System.out.println("PaymentDAO.createPayment error: " + e.getMessage());
36 |              return false;
37 |          }
38 |      }
39 |
40 |      /** Update an existing payment record when partial or full payment is received */
41 |      public boolean updatePayment(Payment payment) {
42 |          String sql = "UPDATE payment SET amount_received=?, remaining_amount=?, payment_date=...
43 |
44 |          try (Connection conn = DatabaseConnection.getConnection();
45 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
46 |
47 |              stmt.setDouble(1, payment.getAmountReceived());
48 |              stmt.setDouble(2, payment.getRemainingAmount());
49 |              stmt.setDate(3, payment.getPaymentDate() != null
50 |                      ? new java.sql.Date(payment.getPaymentDate().getTime()) : null);
51 |              stmt.setString(4, payment.getPaymentStatus());
52 |              stmt.setString(5, payment.getNotes());
53 |              stmt.setInt(6, payment.getId());
54 |
55 |              return stmt.executeUpdate() > 0;
56 |
57 |          } catch (SQLException e) {
58 |              System.out.println("PaymentDAO.updatePayment error: " + e.getMessage());
59 |              return false;
60 |          }
61 |      }
62 |
63 |      /** Calculate total pending unpaid balance across all invoices */
64 |      public double getTotalPendingAmount() {
65 |          String sql = "SELECT COALESCE(SUM(remaining_amount), 0) FROM payment WHERE payment_st...
66 |          try (Connection conn = DatabaseConnection.getConnection();
67 |               PreparedStatement stmt = conn.prepareStatement(sql);
68 |               ResultSet rs = stmt.executeQuery()) {
69 |
70 |              if (rs.next()) return rs.getDouble(1);
71 |
72 |          } catch (SQLException e) {
73 |              System.out.println("PaymentDAO.getTotalPendingAmount error: " + e.getMessage());
74 |          }
75 |          return 0.0;
76 |      }
77 |
78 |      /** Retrieve payment records for a specific company */
79 |      public List<Payment> getPaymentsByCompany(int companyId) {
80 |          List<Payment> payments = new ArrayList<>();
81 |          String sql = "SELECT p.*, i.invoice_number, c.company_name " +
82 |                       "FROM payment p " +
83 |                       "JOIN invoice i ON p.invoice_id = i.id " +
84 |                       "JOIN company c ON p.company_id = c.id " +
85 |                       "WHERE p.company_id = ? " +
86 |                       "ORDER BY p.id DESC";
87 |
88 |          try (Connection conn = DatabaseConnection.getConnection();
89 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
90 |
91 |              stmt.setInt(1, companyId);
92 |              ResultSet rs = stmt.executeQuery();
93 |              while (rs.next()) {
94 |                  payments.add(mapRow(rs));
95 |              }
96 |
97 |          } catch (SQLException e) {
98 |              System.out.println("PaymentDAO.getPaymentsByCompany error: " + e.getMessage());
99 |          }
100 |         return payments;
101 |     }
102 |
103 |     /** Retrieve all payment records */
104 |     public List<Payment> getAllPayments() {
```

```
105 |          List<Payment> payments = new ArrayList<>();
106 |          String sql = "SELECT p.*, i.invoice_number, c.company_name " +
107 |                       "FROM payment p " +
108 |                       "JOIN invoice i ON p.invoice_id = i.id " +
109 |                       "JOIN company c ON p.company_id = c.id " +
110 |                       "ORDER BY p.id DESC";
111 |
112 |          try (Connection conn = DatabaseConnection.getConnection();
113 |               PreparedStatement stmt = conn.prepareStatement(sql);
114 |               ResultSet rs = stmt.executeQuery()) {
115 |
116 |              while (rs.next()) {
117 |                  payments.add(mapRow(rs));
118 |              }
119 |
120 |          } catch (SQLException e) {
121 |              System.out.println("PaymentDAO.getAllPayments error: " + e.getMessage());
122 |          }
123 |          return payments;
124 |      }
125 |
126 |      /** Retrieve payment record by invoice ID */
127 |      public Payment getPaymentByInvoiceId(int invoiceId) {
128 |          String sql = "SELECT p.*, i.invoice_number, c.company_name " +
129 |                       "FROM payment p " +
130 |                       "JOIN invoice i ON p.invoice_id = i.id " +
131 |                       "JOIN company c ON p.company_id = c.id " +
132 |                       "WHERE p.invoice_id=?";
133 |
134 |          try (Connection conn = DatabaseConnection.getConnection();
135 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
136 |
137 |              stmt.setInt(1, invoiceId);
138 |              ResultSet rs = stmt.executeQuery();
139 |              if (rs.next()) return mapRow(rs);
140 |
141 |          } catch (SQLException e) {
142 |              System.out.println("PaymentDAO.getPaymentByInvoiceId error: " + e.getMessage());
143 |          }
144 |          return null;
145 |      }
146 |
147 |      /** Calculate total pending unpaid balance for a company */
148 |      public double getPendingAmountByCompany(int companyId) {
149 |          String sql = "SELECT COALESCE(SUM(remaining_amount), 0) FROM payment WHERE company_id...
150 |          try (Connection conn = DatabaseConnection.getConnection();
151 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
152 |
153 |              stmt.setInt(1, companyId);
154 |              ResultSet rs = stmt.executeQuery();
155 |              if (rs.next()) return rs.getDouble(1);
156 |
157 |          } catch (SQLException e) {
158 |              System.out.println("PaymentDAO.getPendingAmountByCompany error: " + e.getMessage());
159 |          }
160 |          return 0.0;
161 |      }
162 |
163 |      /** Helper: convert ResultSet row into a Payment object */
164 |      private Payment mapRow(ResultSet rs) throws SQLException {
165 |          Payment p = new Payment();
166 |          p.setId(rs.getInt("id"));
167 |          p.setInvoiceId(rs.getInt("invoice_id"));
168 |          p.setCompanyId(rs.getInt("company_id"));
169 |          p.setInvoiceNumber(rs.getString("invoice_number"));
170 |          p.setCompanyName(rs.getString("company_name"));
171 |          p.setTotalAmount(rs.getDouble("total_amount"));
172 |          p.setAmountReceived(rs.getDouble("amount_received"));
173 |          p.setRemainingAmount(rs.getDouble("remaining_amount"));
174 |          p.setPaymentDate(rs.getDate("payment_date"));
175 |          p.setPaymentStatus(rs.getString("payment_status"));
176 |          p.setNotes(rs.getString("notes"));
177 |          return p;
178 |      }
179 |  }
```

FILE: src/com/transport/dao/VehicleDAO.java

[DAO] | 195 lines

Fleet Vehicle Registration & Assignment DAO

```
1 |  package com.transport.dao;
2 |
3 |  import com.transport.db.DatabaseConnection;
4 |  import com.transport.model.Vehicle;
5 |  import java.sql.*;
6 |  import java.util.ArrayList;
7 |  import java.util.List;
8 |
9 |  /**
```

```
10 |    * VehicleDAO.java - Data Access Object for Vehicle
11 |    *
12 |    * Handles vehicle operations including:
13 |    * - Driver fleet management (vehicles owned/operated by a driver)
14 |    * - Company-specific fleet visibility (vehicles engaged by a company)
15 |    * - Vehicle registrations and assignments
16 |    */
17 |   public class VehicleDAO {
18 |
19 |       /** Add a new vehicle to the database */
20 |       public boolean addVehicle(Vehicle vehicle) {
21 |           String sql = "INSERT INTO vehicle (registration_number, vehicle_type, driver_id, stat...
22 |
23 |           try (Connection conn = DatabaseConnection.getConnection();
24 |                PreparedStatement stmt = conn.prepareStatement(sql)) {
25 |
26 |               stmt.setString(1, vehicle.getRegistrationNumber());
27 |               stmt.setString(2, vehicle.getVehicleType());
28 |               if (vehicle.getDriverId() > 0) {
29 |                   stmt.setInt(3, vehicle.getDriverId());
30 |               } else {
31 |                   stmt.setNull(3, Types.INTEGER);
32 |               }
33 |               stmt.setString(4, vehicle.getStatus() != null ? vehicle.getStatus() : "Active");
34 |
35 |               return stmt.executeUpdate() > 0;
36 |
37 |           } catch (SQLException e) {
38 |               System.out.println("VehicleDAO.addVehicle error: " + e.getMessage());
39 |               return false;
40 |           }
41 |       }
42 |
43 |       /** Update existing vehicle details */
44 |       public boolean updateVehicle(Vehicle vehicle) {
45 |           String sql = "UPDATE vehicle SET registration_number=?, vehicle_type=?, driver_id=?, ...
46 |
47 |           try (Connection conn = DatabaseConnection.getConnection();
48 |                PreparedStatement stmt = conn.prepareStatement(sql)) {
49 |
50 |               stmt.setString(1, vehicle.getRegistrationNumber());
51 |               stmt.setString(2, vehicle.getVehicleType());
52 |               if (vehicle.getDriverId() > 0) {
53 |                   stmt.setInt(3, vehicle.getDriverId());
54 |               } else {
55 |                   stmt.setNull(3, Types.INTEGER);
56 |               }
57 |               stmt.setString(4, vehicle.getStatus());
58 |               stmt.setInt(5, vehicle.getId());
59 |
60 |               return stmt.executeUpdate() > 0;
61 |
62 |           } catch (SQLException e) {
63 |               System.out.println("VehicleDAO.updateVehicle error: " + e.getMessage());
64 |               return false;
65 |           }
66 |       }
67 |
68 |       /** Delete a vehicle by ID */
69 |       public boolean deleteVehicle(int id) {
70 |           String sql = "DELETE FROM vehicle WHERE id=?";
71 |
72 |           try (Connection conn = DatabaseConnection.getConnection();
73 |                PreparedStatement stmt = conn.prepareStatement(sql)) {
74 |
75 |               stmt.setInt(1, id);
76 |               return stmt.executeUpdate() > 0;
77 |
78 |           } catch (SQLException e) {
79 |               System.out.println("VehicleDAO.deleteVehicle error: " + e.getMessage());
80 |               return false;
81 |           }
82 |       }
83 |
84 |       /** Retrieve all vehicles */
85 |       public List<Vehicle> getAllVehicles() {
86 |           List<Vehicle> vehicles = new ArrayList<>();
87 |           String sql = "SELECT v.*, d.driver_name FROM vehicle v LEFT JOIN driver d ON v.driver...
88 |
89 |           try (Connection conn = DatabaseConnection.getConnection();
90 |                PreparedStatement stmt = conn.prepareStatement(sql);
91 |                ResultSet rs = stmt.executeQuery()) {
92 |
93 |               while (rs.next()) {
94 |                   vehicles.add(mapRow(rs));
95 |               }
96 |
97 |           } catch (SQLException e) {
98 |               System.out.println("VehicleDAO.getAllVehicles error: " + e.getMessage());
```

```
        }
        return vehicles;
    }

    /** Retrieve vehicles owned or operated by a specific driver */
    public List<Vehicle> getVehiclesByDriver(int driverId) {
        List<Vehicle> vehicles = new ArrayList<>();
        String sql = "SELECT v.*, d.driver_name FROM vehicle v " +
                     "LEFT JOIN driver d ON v.driver_id = d.id " +
                     "WHERE v.driver_id = ? " +
                     "ORDER BY v.registration_number";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, driverId);
            ResultSet rs = stmt.executeQuery();
            while (rs.next()) {
                vehicles.add(mapRow(rs));
            }

        } catch (SQLException e) {
            System.out.println("VehicleDAO.getVehiclesByDriver error: " + e.getMessage());
        }
        return vehicles;
    }

    /** Retrieve vehicles used in trips for a specific company */
    public List<Vehicle> getVehiclesByCompany(int companyId) {
        List<Vehicle> vehicles = new ArrayList<>();
        String sql = "SELECT DISTINCT v.*, d.driver_name FROM trip t " +
                     "JOIN vehicle v ON t.vehicle_id = v.id " +
                     "LEFT JOIN driver d ON v.driver_id = d.id " +
                     "WHERE t.company_id = ? " +
                     "ORDER BY v.registration_number";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, companyId);
            ResultSet rs = stmt.executeQuery();
            while (rs.next()) {
                vehicles.add(mapRow(rs));
            }

        } catch (SQLException e) {
            System.out.println("VehicleDAO.getVehiclesByCompany error: " + e.getMessage());
        }
        return vehicles;
    }

    /** Retrieve a vehicle by its ID */
    public Vehicle getVehicleById(int id) {
        String sql = "SELECT v.*, d.driver_name FROM vehicle v LEFT JOIN driver d ON v.driver...

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, id);
            ResultSet rs = stmt.executeQuery();
            if (rs.next()) return mapRow(rs);

        } catch (SQLException e) {
            System.out.println("VehicleDAO.getVehicleById error: " + e.getMessage());
        }
        return null;
    }

    /** Retrieve a vehicle by registration number */
    public Vehicle getVehicleByRegNo(String regNo) {
        String sql = "SELECT v.*, d.driver_name FROM vehicle v LEFT JOIN driver d ON v.driver...

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, regNo.trim());
            ResultSet rs = stmt.executeQuery();
            if (rs.next()) return mapRow(rs);

        } catch (SQLException e) {
            System.out.println("VehicleDAO.getVehicleByRegNo error: " + e.getMessage());
        }
        return null;
    }

    /** Helper: convert a ResultSet row into a Vehicle object */
    private Vehicle mapRow(ResultSet rs) throws SQLException {
        Vehicle v = new Vehicle();
        v.setId(rs.getInt("id"));
```

```
        v.setRegistrationNumber(rs.getString("registration_number"));
        v.setVehicleType(rs.getString("vehicle_type"));
        v.setDriverId(rs.getInt("driver_id"));
        v.setDriverName(rs.getString("driver_name") != null ? rs.getString("driver_name") : "...
        v.setStatus(rs.getString("status"));
        return v;
    }
}
```

FILE: src/com/transport/dao/AdminDAO.java **[DAO] | 180 lines**

System Admin Authentication & Recovery DAO

```
package com.transport.dao;

import com.transport.db.DatabaseConnection;
import com.transport.model.Admin;
import com.transport.model.Company;
import java.sql.*;

/**
 * AdminDAO.java - Data Access Object for Company Administrator
 *
 * Implements authentication and profile management for Company Admins.
 * Each company has its own independent login credentials stored in the 'company' table.
 */
public class AdminDAO {

    /**
     * Authenticates a Company Admin using either Username OR Email along with Password.
     * Returns an Admin object containing company details if successful, null otherwise.
     */
    public Admin login(String usernameOrEmail, String password) {
        String sql = "SELECT * FROM company WHERE (username = ? OR email = ?) AND password = ?";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, usernameOrEmail != null ? usernameOrEmail.trim() : "");
            stmt.setString(2, usernameOrEmail != null ? usernameOrEmail.trim() : "");
            stmt.setString(3, password != null ? password.trim() : "");

            ResultSet rs = stmt.executeQuery();

            if (rs.next()) {
                Admin admin = new Admin();
                admin.setId(rs.getInt("id"));
                admin.setCompanyId(rs.getInt("id"));
                admin.setCompanyName(rs.getString("company_name"));
                admin.setAdminName(rs.getString("admin_name"));
                admin.setEmail(rs.getString("email"));
                admin.setUsername(rs.getString("username"));
                admin.setPassword(rs.getString("password"));
                admin.setContactNumber(rs.getString("contact_number"));
                admin.setAddress(rs.getString("address"));
                admin.setGstNumber(rs.getString("gst_number"));
                return admin;
            }

        } catch (SQLException e) {
            System.out.println("AdminDAO.login error: " + e.getMessage());
        }

        return null;
    }

    /**
     * Register a new company account.
     */
    public boolean registerCompany(Company company) {
        String sql = "INSERT INTO company (company_name, admin_name, email, username, passwor...
                     "VALUES (?, ?, ?, ?, ?, ?, ?, ?)";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, company.getCompanyName());
            stmt.setString(2, company.getAdminName());
            stmt.setString(3, company.getEmail());
            stmt.setString(4, company.getUsername());
            stmt.setString(5, company.getPassword());
            stmt.setString(6, company.getContactNumber());
            stmt.setString(7, company.getAddress());
            stmt.setString(8, company.getGstNumber());

            return stmt.executeUpdate() > 0;

        } catch (SQLException e) {
            System.out.println("AdminDAO.registerCompany error: " + e.getMessage());
```

```
77 |              return false;
78 |          }
79 |      }
80 |
81 |      /**
82 |       * Updates company profile details (contact, address, admin name, gst, etc.)
83 |       */
84 |      public boolean updateCompanyProfile(Admin admin) {
85 |          String sql = "UPDATE company SET company_name=?, admin_name=?, email=?, contact_numbe...
86 |
87 |          try (Connection conn = DatabaseConnection.getConnection();
88 |               PreparedStatement stmt = conn.prepareStatement(sql)) {
89 |
90 |              stmt.setString(1, admin.getCompanyName());
91 |              stmt.setString(2, admin.getAdminName());
92 |              stmt.setString(3, admin.getEmail());
93 |              stmt.setString(4, admin.getContactNumber());
94 |              stmt.setString(5, admin.getAddress());
95 |              stmt.setString(6, admin.getGstNumber());
96 |              stmt.setInt(7, admin.getCompanyId());
97 |
98 |              return stmt.executeUpdate() > 0;
99 |
100|          } catch (SQLException e) {
101|              System.out.println("AdminDAO.updateCompanyProfile error: " + e.getMessage());
102|              return false;
103|          }
104|      }
105|
106|      /**
107|       * Updates password for a company admin.
108|       */
109|      public boolean updatePassword(int companyId, String newPassword) {
110|          String sql = "UPDATE company SET password=? WHERE id=?";
111|
112|          try (Connection conn = DatabaseConnection.getConnection();
113|               PreparedStatement stmt = conn.prepareStatement(sql)) {
114|
115|              stmt.setString(1, newPassword);
116|              stmt.setInt(2, companyId);
117|
118|              return stmt.executeUpdate() > 0;
119|
120|          } catch (SQLException e) {
121|              System.out.println("AdminDAO.updatePassword error: " + e.getMessage());
122|              return false;
123|          }
124|      }
125|
126|      /**
127|       * Reset password using username or email (Forgot Password recovery).
128|       */
129|      public boolean resetPassword(String usernameOrEmail, String newPassword) {
130|          String sql = "UPDATE company SET password=? WHERE username=? OR email=?";
131|
132|          try (Connection conn = DatabaseConnection.getConnection();
133|               PreparedStatement stmt = conn.prepareStatement(sql)) {
134|
135|              stmt.setString(1, newPassword);
136|              stmt.setString(2, usernameOrEmail);
137|              stmt.setString(3, usernameOrEmail);
138|
139|              return stmt.executeUpdate() > 0;
140|
141|          } catch (SQLException e) {
142|              System.out.println("AdminDAO.resetPassword error: " + e.getMessage());
143|              return false;
144|          }
145|      }
146|
147|      /**
148|       * Retrieve company admin details by company ID.
149|       */
150|      public Admin getCompanyById(int companyId) {
151|          String sql = "SELECT * FROM company WHERE id = ?";
152|
153|          try (Connection conn = DatabaseConnection.getConnection();
154|               PreparedStatement stmt = conn.prepareStatement(sql)) {
155|
156|              stmt.setInt(1, companyId);
157|              ResultSet rs = stmt.executeQuery();
158|
159|              if (rs.next()) {
160|                  Admin admin = new Admin();
161|                  admin.setId(rs.getInt("id"));
162|                  admin.setCompanyId(rs.getInt("id"));
163|                  admin.setCompanyName(rs.getString("company_name"));
164|                  admin.setAdminName(rs.getString("admin_name"));
165|                  admin.setEmail(rs.getString("email"));
```

```
                admin.setUsername(rs.getString("username"));
                admin.setPassword(rs.getString("password"));
                admin.setContactNumber(rs.getString("contact_number"));
                admin.setAddress(rs.getString("address"));
                admin.setGstNumber(rs.getString("gst_number"));
                return admin;
            }

        } catch (SQLException e) {
            System.out.println("AdminDAO.getCompanyById error: " + e.getMessage());
        }

        return null;
    }
}
```

FILE: src/com/transport/util/InvoiceGenerator.java **[ENGINE] | 786 lines**

Core Java PDF 1.4 Binary Generator (Invoices, Receipts & Reports)

```
package com.transport.util;

import com.transport.model.Invoice;
import com.transport.model.Trip;

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

/**
 * InvoiceGenerator.java - Native PDF & Text Invoice/Receipt Engine for MOT Logistics
 *
 * Pure Core Java implementation (no third-party JARs required).
 * Generates standards-compliant PDF 1.4 documents (invoices, receipts, and summary ledgers)
 * and formatted text/CSV vouchers.
 */
public class InvoiceGenerator {

    private static final String SERVICE_NAME = "MOT Logistics";

    private static final String[] MONTH_NAMES = {
        "All Months", "January","February","March","April","May","June",
        "July","August","September","October","November","December"
    };

    /** Get user-friendly billing period text */
    public static String getBillingPeriodText(int month, int year) {
        if (month <= 0 || month > 12) {
            return "All Months (Annual Statement " + year + ")";
        }
        return MONTH_NAMES[month] + " " + year;
    }

    /**
     * Generate a multi-page, genuine standard PDF 1.4 invoice document.
     * Compatible with Adobe Acrobat, Apple Preview, Google Chrome, and web browsers.
     *
     * @param invoice  The invoice metadata object
     * @param trips    The itemized list of trips (handles any number of trips across pages)
     * @param filePath The destination file path (.pdf)
     * @return true if successful, false on error
     */
    public static boolean generatePDF(Invoice invoice, List<Trip> trips, String filePath) {
        if (!filePath.toLowerCase().endsWith(".pdf")) {
            filePath += ".pdf";
        }

        ensureDirectoryExists(filePath);
        SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");

        try (ByteArrayOutputStream pdfStream = new ByteArrayOutputStream()) {
            List<Integer> offsets = new ArrayList<>();
            offsets.add(0);

            // Calculate pagination
            int tripsPerPage = 20;
            int totalTrips = (trips != null) ? trips.size() : 0;
            int totalPages = totalTrips <= tripsPerPage ? 1 : (int) Math.ceil((double) totalT...
            if (totalPages == 0) totalPages = 1;

            // 1. PDF Header
            pdfStream.write("%PDF-1.4\n%\u00E2\u00E3\u00CF\u00D3\n".getBytes(StandardCharsets...

            // Catalog (Obj 1)
            offsets.add(pdfStream.size());
            pdfStream.write("1 0 obj\n<< /Type /Catalog /Pages 2 0 R >>\nendobj\n".getBytes(S...
```

```
            // Pages (Obj 2)
            StringBuilder kids = new StringBuilder();
            for (int p = 0; p < totalPages; p++) {
                kids.append((3 + p * 2)).append(" 0 R ");
            }
            offsets.add(pdfStream.size());
            pdfStream.write(String.format("2 0 obj\n<< /Type /Pages /Kids [%s] /Count %d >>\n...
                    kids.toString().trim(), totalPages).getBytes(StandardCharsets.ISO_8859_1));

            int font1Obj = 3 + totalPages * 2;
            int font2Obj = font1Obj + 1;

            double grandTotal = 0.0;
            if (trips != null) {
                for (Trip t : trips) grandTotal += t.getAmount();
            }

            int tripIndex = 0;

            for (int p = 0; p < totalPages; p++) {
                int pageObj = 3 + p * 2;
                int contentObj = pageObj + 1;

                StringBuilder sb = new StringBuilder();

                // Top Navy Brand Header Banner
                sb.append("0.06 0.15 0.33 rg\n");
                sb.append("40 760 515 52 re f\n");

                sb.append("1 1 1 rg\n");
                sb.append("BT /F1 18 Tf 55 790 Td (MOT LOGISTICS) Tj ET\n");
                sb.append("BT /F2 9 Tf 55 772 Td (COMMERCIAL TRANSPORT FLEET MANAGEMENT & BIL...

                // Page Indicator
                sb.append("BT /F2 8 Tf 480 772 Td (Page ").append(p + 1).append(" of ").appen...

                // Invoice Header Details Grid
                sb.append("0.1 0.1 0.1 rg\n");
                sb.append("BT /F1 10 Tf 45 734 Td (Invoice Number:) Tj ET\n");
                sb.append("BT /F2 10 Tf 140 734 Td (").append(cleanPdfText(invoice.getInvoice...

                sb.append("BT /F1 10 Tf 45 716 Td (Invoice Date:) Tj ET\n");
                sb.append("BT /F2 10 Tf 140 716 Td (").append(sdf.format(invoice.getInvoiceDa...

                String periodText = getBillingPeriodText(invoice.getBillingMonth(), invoice.g...
                sb.append("BT /F1 10 Tf 45 698 Td (Billing Period:) Tj ET\n");
                sb.append("BT /F2 10 Tf 140 698 Td (").append(cleanPdfText(periodText)).appen...

                String compBilled = invoice.getCompanyName() != null ? invoice.getCompanyName...
                if (compBilled.length() > 24) compBilled = compBilled.substring(0, 22) + "..";
                sb.append("BT /F1 10 Tf 315 734 Td (Billed To:) Tj ET\n");
                sb.append("BT /F2 10 Tf 385 734 Td (").append(cleanPdfText(compBilled)).appen...

                String driverText = (invoice.getDriverName() != null && !invoice.getDriverNam...
                        ? invoice.getDriverName() : "All / Fleet Operations";
                if (driverText.length() > 24) driverText = driverText.substring(0, 22) + "..";
                sb.append("BT /F1 10 Tf 315 716 Td (Driver:) Tj ET\n");
                sb.append("BT /F2 10 Tf 385 716 Td (").append(cleanPdfText(driverText)).appen...

                String vehicleText = (invoice.getVehicleNumber() != null && !invoice.getVehic...
                        ? invoice.getVehicleNumber() : "Fleet Operations";
                if (vehicleText.length() > 24) vehicleText = vehicleText.substring(0, 22) + "...
                sb.append("BT /F1 10 Tf 315 698 Td (Vehicle:) Tj ET\n");
                sb.append("BT /F2 10 Tf 385 698 Td (").append(cleanPdfText(vehicleText)).appe...

                // Rule
                sb.append("0.85 0.88 0.92 rg\n");
                sb.append("40 682 515 1 re f\n");

                // Itemized Table Header Bar
                sb.append("0.93 0.95 0.98 rg\n");
                sb.append("40 652 515 22 re f\n");
                sb.append("0.06 0.15 0.33 rg\n");
                sb.append("BT /F1 9 Tf 46 659 Td (Sr.) Tj ET\n");
                sb.append("BT /F1 9 Tf 68 659 Td (Trip Date) Tj ET\n");
                sb.append("BT /F1 9 Tf 120 659 Td (Driver Name) Tj ET\n");
                sb.append("BT /F1 9 Tf 235 659 Td (Vehicle Reg.) Tj ET\n");
                sb.append("BT /F1 9 Tf 310 659 Td (Route (From -> To)) Tj ET\n");
                sb.append("BT /F1 9 Tf 470 659 Td (Amount (INR)) Tj ET\n");

                int y = 632;
                int pageTripsRendered = 0;

                while (trips != null && tripIndex < trips.size() && pageTripsRendered < trips...
                    Trip trip = trips.get(tripIndex);

                    // Row divider line
                    sb.append("0.92 0.94 0.96 rg\n");
```

```
                    sb.append("40 ").append(y - 5).append(" 515 0.5 re f\n");

                    String dName = trip.getDriverName() != null ? trip.getDriverName() : "-";
                    if (dName.length() > 22) dName = dName.substring(0, 20) + "..";
                    String vNo   = trip.getVehicleNumber() != null ? trip.getVehicleNumber() ...
                    if (vNo.length() > 14) vNo = vNo.substring(0, 12) + "..";
                    String route = trip.getSourceLocation() + " -> " + trip.getDestination();
                    if (route.length() > 32) route = route.substring(0, 30) + "..";

                    sb.append("0.15 0.15 0.15 rg\n");
                    sb.append("BT /F2 8 Tf 46 ").append(y).append(" Td (").append(tripIndex +...
                    sb.append("BT /F2 8 Tf 68 ").append(y).append(" Td (").append(sdf.format(...
                    sb.append("BT /F2 8 Tf 120 ").append(y).append(" Td (").append(cleanPdfTe...
                    sb.append("BT /F2 8 Tf 235 ").append(y).append(" Td (").append(cleanPdfTe...
                    sb.append("BT /F2 8 Tf 310 ").append(y).append(" Td (").append(cleanPdfTe...
                    sb.append("BT /F2 8 Tf 470 ").append(y).append(" Td (").append(String.for...

                    y -= 20;
                    tripIndex++;
                    pageTripsRendered++;
                }

                // If last page, render Summary & Signatures
                if (p == totalPages - 1) {
                    y -= 6;
                    sb.append("0.95 0.97 0.99 rg\n");
                    sb.append("40 ").append(y - 8).append(" 515 26 re f\n");

                    sb.append("0.06 0.15 0.33 rg\n");
                    sb.append("BT /F1 10 Tf 50 ").append(y).append(" Td (Total Completed Trip...
                    sb.append("BT /F1 11 Tf 340 ").append(y).append(" Td (TOTAL PAYABLE: INR ...

                    int sigY = Math.max(90, y - 80);
                    sb.append("0.2 0.2 0.2 rg\n");
                    sb.append("40 ").append(sigY + 30).append(" 180 1 re f\n");
                    sb.append("BT /F1 9 Tf 40 ").append(sigY + 15).append(" Td (Authorized Si...
                    sb.append("BT /F2 8 Tf 40 ").append(sigY + 3).append(" Td (MOT Logistics)...

                    sb.append("375 ").append(sigY + 30).append(" 180 1 re f\n");
                    sb.append("BT /F1 9 Tf 375 ").append(sigY + 15).append(" Td (Client Signa...
                    sb.append("BT /F2 8 Tf 375 ").append(sigY + 3).append(" Td (").append(cle...
                }

                // Footer
                sb.append("0.5 0.5 0.5 rg\n");
                sb.append("40 45 515 0.5 re f\n");
                sb.append("BT /F2 8 Tf 40 32 Td (Generated on: ").append(new Date().toString())
                  .append(" | Computer generated invoice by MOT Logistics) Tj ET\n");

                byte[] streamBytes = sb.toString().getBytes(StandardCharsets.ISO_8859_1);

                // Write Page Object
                offsets.add(pdfStream.size());
                pdfStream.write(String.format("%d 0 obj\n<< /Type /Page /Parent 2 0 R /MediaB...
                        pageObj, contentObj, font1Obj, font2Obj).getBytes(StandardCharsets.IS...

                // Write Content Stream Object
                offsets.add(pdfStream.size());
                pdfStream.write(String.format("%d 0 obj\n<< /Length %d >>\nstream\n", content...
                pdfStream.write(streamBytes);
                pdfStream.write("\nendstream\nendobj\n".getBytes(StandardCharsets.ISO_8859_1));
            }

            // Fonts
            offsets.add(pdfStream.size());
            pdfStream.write(String.format("%d 0 obj\n<< /Type /Font /Subtype /Type1 /BaseFont...

            offsets.add(pdfStream.size());
            pdfStream.write(String.format("%d 0 obj\n<< /Type /Font /Subtype /Type1 /BaseFont...

            // Cross-reference Table (xref)
            int xrefStart = pdfStream.size();
            pdfStream.write(String.format("xref\n0 %d\n0000000000 65535 f \n", offsets.size()...
            for (int i = 1; i < offsets.size(); i++) {
                pdfStream.write(String.format("%010d 00000 n \n", offsets.get(i)).getBytes(St...
            }

            // Trailer & EOF
            pdfStream.write(String.format("trailer\n<< /Size %d /Root 1 0 R >>\nstartxref\n%d...
                    offsets.size(), xrefStart).getBytes(StandardCharsets.ISO_8859_1));

            // Output to file
            try (FileOutputStream fos = new FileOutputStream(new File(filePath))) {
                pdfStream.writeTo(fos);
            }

            System.out.println("PDF Invoice generated successfully: " + filePath);
            return true;

```

```
248 |           } catch (Exception e) {
249 |               System.err.println("InvoiceGenerator.generatePDF error: " + e.getMessage());
250 |               e.printStackTrace();
251 |               return false;
252 |           }
253 |       }
254 |
255 |       /**
256 |        * Generate an official Single Trip Receipt / Cash Voucher in PDF format.
257 |        */
258 |       public static boolean generateTripReceiptPDF(Trip trip, String filePath) {
259 |           if (!filePath.toLowerCase().endsWith(".pdf")) {
260 |               filePath += ".pdf";
261 |           }
262 |           ensureDirectoryExists(filePath);
263 |           SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
264 |
265 |           try (ByteArrayOutputStream pdfStream = new ByteArrayOutputStream()) {
266 |               List<Integer> offsets = new ArrayList<>();
267 |               offsets.add(0);
268 |
269 |               pdfStream.write("%PDF-1.4\n%\u00E2\u00E3\u00CF\u00D3\n".getBytes(StandardCharsets...
270 |
271 |               // Catalog (Obj 1)
272 |               offsets.add(pdfStream.size());
273 |               pdfStream.write("1 0 obj\n<< /Type /Catalog /Pages 2 0 R >>\nendobj\n".getBytes(S...
274 |
275 |               // Pages (Obj 2)
276 |               offsets.add(pdfStream.size());
277 |               pdfStream.write("2 0 obj\n<< /Type /Pages /Kids [3 0 R] /Count 1 >>\nendobj\n".ge...
278 |
279 |               // Page (Obj 3)
280 |               offsets.add(pdfStream.size());
281 |               pdfStream.write("3 0 obj\n<< /Type /Page /Parent 2 0 R /MediaBox [0 0 595 842] /C...
282 |
283 |               StringBuilder sb = new StringBuilder();
284 |
285 |               // Brand Banner
286 |               sb.append("0.06 0.15 0.33 rg\n");
287 |               sb.append("40 760 515 52 re f\n");
288 |               sb.append("1 1 1 rg\n");
289 |               sb.append("BT /F1 18 Tf 55 790 Td (MOT LOGISTICS) Tj ET\n");
290 |               sb.append("BT /F2 9 Tf 55 772 Td (OFFICIAL TRIP RECEIPT & DELIVERY VOUCHER) Tj ET...
291 |
292 |               // Receipt Box
293 |               sb.append("0.96 0.97 0.99 rg\n");
294 |               sb.append("40 440 515 300 re f\n");
295 |               sb.append("0.8 0.85 0.9 rg\n");
296 |               sb.append("40 440 515 300 re s\n");
297 |
298 |               sb.append("0.1 0.1 0.1 rg\n");
299 |               sb.append("BT /F1 12 Tf 55 715 Td (Receipt No: REC-").append(cleanPdfText(trip.ge...
300 |               sb.append("BT /F1 10 Tf 370 715 Td (Trip Date: ").append(sdf.format(trip.getTripD...
301 |
302 |               String driverName = trip.getDriverName();
303 |               if ((driverName == null || driverName.trim().isEmpty() || driverName.equals("-"))...
304 |                   try {
305 |                       com.transport.model.Driver d = new com.transport.dao.DriverDAO().getDrive...
306 |                       if (d != null && d.getDriverName() != null && !d.getDriverName().trim().i...
307 |                           driverName = d.getDriverName();
308 |                       }
309 |                   } catch (Exception ignored) {}
310 |               }
311 |               if (driverName == null || driverName.trim().isEmpty()) driverName = "Fleet Driver";
312 |
313 |               String compName = trip.getCompanyName();
314 |               if ((compName == null || compName.trim().isEmpty() || compName.equals("-")) && tr...
315 |                   try {
316 |                       com.transport.model.Company c = new com.transport.dao.CompanyDAO().getCom...
317 |                       if (c != null && c.getCompanyName() != null && !c.getCompanyName().trim()...
318 |                           compName = c.getCompanyName();
319 |                       }
320 |                   } catch (Exception ignored) {}
321 |               }
322 |               if (compName == null || compName.trim().isEmpty()) compName = "Direct Cargo Client";
323 |
324 |               String vehNum = trip.getVehicleNumber();
325 |               if ((vehNum == null || vehNum.trim().isEmpty() || vehNum.equals("-")) && trip.get...
326 |                   try {
327 |                       com.transport.model.Vehicle v = new com.transport.dao.VehicleDAO().getVeh...
328 |                       if (v != null && v.getRegistrationNumber() != null && !v.getRegistrationN...
329 |                           vehNum = v.getRegistrationNumber();
330 |                       }
331 |                   } catch (Exception ignored) {}
332 |               }
333 |               if (vehNum == null || vehNum.trim().isEmpty()) vehNum = "Fleet Commercial Vehicle";
334 |
335 |               int y = 670;
336 |               String[][] fields = {
```

```
                {"Trip Reference Code", trip.getTripCode()},
                {"Client / Consignor",  compName},
                {"Driver In-Charge (Full Name)", driverName},
                {"Vehicle Reg. Number", vehNum},
                {"Origin (Pickup)",     trip.getSourceLocation()},
                {"Destination (Drop)",  trip.getDestination()},
                {"Remarks / Consignment Notes", trip.getRemarks() != null && !trip.getRemarks...
            };

            for (String[] row : fields) {
                String val = row[1] != null ? row[1] : "-";
                if (val.length() > 50) val = val.substring(0, 48) + "..";
                sb.append("BT /F1 10 Tf 55 ").append(y).append(" Td (").append(cleanPdfText(r...
                sb.append("BT /F2 10 Tf 220 ").append(y).append(" Td (").append(cleanPdfText(...
                sb.append("0.88 0.9 0.94 rg\n");
                sb.append("55 ").append(y - 6).append(" 480 0.5 re f\n");
                sb.append("0.1 0.1 0.1 rg\n");
                y -= 26;
            }

            // Fare Amount Highlight Banner
            y -= 10;
            sb.append("0.06 0.15 0.33 rg\n");
            sb.append("55 ").append(y - 12).append(" 480 34 re f\n");
            sb.append("1 1 1 rg\n");
            sb.append("BT /F1 12 Tf 70 ").append(y).append(" Td (TOTAL TRIP FARE / VOUCHER AM...
            sb.append("BT /F1 14 Tf 370 ").append(y).append(" Td (INR ").append(String.format...

            // Signatures
            int sigY = 320;
            sb.append("0.2 0.2 0.2 rg\n");
            sb.append("55 ").append(sigY + 30).append(" 180 1 re f\n");
            sb.append("BT /F1 9 Tf 55 ").append(sigY + 15).append(" Td (Driver Signature) Tj ...
            sb.append("BT /F2 9 Tf 55 ").append(sigY + 3).append(" Td (").append(cleanPdfText...

            sb.append("355 ").append(sigY + 30).append(" 180 1 re f\n");
            sb.append("BT /F1 9 Tf 355 ").append(sigY + 15).append(" Td (Client / Receiver Ac...
            sb.append("BT /F2 9 Tf 355 ").append(sigY + 3).append(" Td (").append(cleanPdfTex...

            // Footer
            sb.append("0.5 0.5 0.5 rg\n");
            sb.append("40 50 515 0.5 re f\n");
            sb.append("BT /F2 8 Tf 40 38 Td (Generated on: ").append(new Date().toString())
              .append(" | Official electronic voucher by MOT Logistics) Tj ET\n");

            byte[] streamBytes = sb.toString().getBytes(StandardCharsets.ISO_8859_1);

            // Content Stream (Obj 4)
            offsets.add(pdfStream.size());
            pdfStream.write(String.format("4 0 obj\n<< /Length %d >>\nstream\n", streamBytes....
            pdfStream.write(streamBytes);
            pdfStream.write("\nendstream\nendobj\n".getBytes(StandardCharsets.ISO_8859_1));

            // Fonts (Obj 5, 6)
            offsets.add(pdfStream.size());
            pdfStream.write("5 0 obj\n<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica-Bol...
            offsets.add(pdfStream.size());
            pdfStream.write("6 0 obj\n<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>\...

            // Xref & Trailer
            int xrefStart = pdfStream.size();
            pdfStream.write(String.format("xref\n0 %d\n0000000000 65535 f \n", offsets.size()...
            for (int i = 1; i < offsets.size(); i++) {
                pdfStream.write(String.format("%010d 00000 n \n", offsets.get(i)).getBytes(St...
            }
            pdfStream.write(String.format("trailer\n<< /Size %d /Root 1 0 R >>\nstartxref\n%d...

            try (FileOutputStream fos = new FileOutputStream(new File(filePath))) {
                pdfStream.writeTo(fos);
            }
            return true;

        } catch (Exception e) {
            System.err.println("InvoiceGenerator.generateTripReceiptPDF error: " + e.getMessa...
            e.printStackTrace();
            return false;
        }
    }

    /**
     * Generate an official Trip Report PDF containing all filtered trips.
     */
    public static boolean generateTripsSummaryPDF(List<Trip> trips, String title, String subt...
        if (!filePath.toLowerCase().endsWith(".pdf")) {
            filePath += ".pdf";
        }
        ensureDirectoryExists(filePath);
        SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");

```

```
426 |           try (ByteArrayOutputStream pdfStream = new ByteArrayOutputStream()) {
427 |               List<Integer> offsets = new ArrayList<>();
428 |               offsets.add(0);
429 |
430 |               int tripsPerPage = 22;
431 |               int totalTrips = (trips != null) ? trips.size() : 0;
432 |               int totalPages = totalTrips <= tripsPerPage ? 1 : (int) Math.ceil((double) totalT...
433 |               if (totalPages == 0) totalPages = 1;
434 |
435 |               pdfStream.write("%PDF-1.4\n%\u00E2\u00E3\u00CF\u00D3\n".getBytes(StandardCharsets...
436 |
437 |               offsets.add(pdfStream.size());
438 |               pdfStream.write("1 0 obj\n<< /Type /Catalog /Pages 2 0 R >>\nendobj\n".getBytes(S...
439 |
440 |               StringBuilder kids = new StringBuilder();
441 |               for (int p = 0; p < totalPages; p++) kids.append((3 + p * 2)).append(" 0 R ");
442 |               offsets.add(pdfStream.size());
443 |               pdfStream.write(String.format("2 0 obj\n<< /Type /Pages /Kids [%s] /Count %d >>\n...
444 |
445 |               int font1Obj = 3 + totalPages * 2;
446 |               int font2Obj = font1Obj + 1;
447 |
448 |               double grandTotal = 0.0;
449 |               if (trips != null) {
450 |                   for (Trip t : trips) grandTotal += t.getAmount();
451 |               }
452 |
453 |               int tripIndex = 0;
454 |
455 |               for (int p = 0; p < totalPages; p++) {
456 |                   int pageObj = 3 + p * 2;
457 |                   int contentObj = pageObj + 1;
458 |
459 |                   StringBuilder sb = new StringBuilder();
460 |                   sb.append("0.06 0.15 0.33 rg\n");
461 |                   sb.append("40 760 515 52 re f\n");
462 |                   sb.append("1 1 1 rg\n");
463 |                   sb.append("BT /F1 16 Tf 55 790 Td (").append(cleanPdfText(title)).append(") T...
464 |                   sb.append("BT /F2 9 Tf 55 772 Td (").append(cleanPdfText(subtitle)).append(")...
465 |                   sb.append("BT /F2 8 Tf 480 772 Td (Page ").append(p + 1).append(" of ").appen...
466 |
467 |                   // Table Header
468 |                   sb.append("0.93 0.95 0.98 rg\n");
469 |                   sb.append("40 725 515 22 re f\n");
470 |                   sb.append("0.06 0.15 0.33 rg\n");
471 |                   sb.append("BT /F1 9 Tf 45 732 Td (Trip) Tj ET\n");
472 |                   sb.append("BT /F1 9 Tf 115 732 Td (Date) Tj ET\n");
473 |                   sb.append("BT /F1 9 Tf 170 732 Td (Company) Tj ET\n");
474 |                   sb.append("BT /F1 9 Tf 270 732 Td (Driver (Full Name)) Tj ET\n");
475 |                   sb.append("BT /F1 9 Tf 375 732 Td (From -> To) Tj ET\n");
476 |                   sb.append("BT /F1 9 Tf 485 732 Td (Amount (INR)) Tj ET\n");
477 |
478 |                   int y = 705;
479 |                   int count = 0;
480 |                   while (trips != null && tripIndex < trips.size() && count < tripsPerPage) {
481 |                       Trip t = trips.get(tripIndex);
482 |                       sb.append("0.92 0.94 0.96 rg\n");
483 |                       sb.append("40 ").append(y - 4).append(" 515 0.5 re f\n");
484 |                       sb.append("0.15 0.15 0.15 rg\n");
485 |
486 |                       String code = t.getTripCode() != null ? t.getTripCode() : "-";
487 |                       if (code.length() > 14) code = code.substring(0, 12) + "..";
488 |                       sb.append("BT /F2 8 Tf 45 ").append(y).append(" Td (").append(cleanPdfTex...
489 |
490 |                       sb.append("BT /F2 8 Tf 115 ").append(y).append(" Td (").append(sdf.format...
491 |
492 |                       String comp = t.getCompanyName() != null ? t.getCompanyName() : "-";
493 |                       if (comp.length() > 20) comp = comp.substring(0, 18) + "..";
494 |                       sb.append("BT /F2 8 Tf 170 ").append(y).append(" Td (").append(cleanPdfTe...
495 |
496 |                       String dname = t.getDriverName() != null ? t.getDriverName() : "-";
497 |                       if (dname.length() > 22) dname = dname.substring(0, 20) + "..";
498 |                       sb.append("BT /F2 8 Tf 270 ").append(y).append(" Td (").append(cleanPdfTe...
499 |
500 |                       String route = t.getSourceLocation() + " -> " + t.getDestination();
501 |                       if (route.length() > 22) route = route.substring(0, 20) + "..";
502 |                       sb.append("BT /F2 8 Tf 375 ").append(y).append(" Td (").append(cleanPdfTe...
503 |
504 |                       sb.append("BT /F2 8 Tf 485 ").append(y).append(" Td (").append(String.for...
505 |
506 |                       y -= 19;
507 |                       tripIndex++;
508 |                       count++;
509 |                   }
510 |
511 |                   if (p == totalPages - 1) {
512 |                       y -= 6;
513 |                       sb.append("0.95 0.97 0.99 rg\n");
514 |                       sb.append("40 ").append(y - 6).append(" 515 24 re f\n");
```

```
                    sb.append("0.06 0.15 0.33 rg\n");
                    sb.append("BT /F1 10 Tf 50 ").append(y).append(" Td (Total Completed Trip...
                    sb.append("BT /F1 11 Tf 340 ").append(y).append(" Td (TOTAL REVENUE: INR ...
                }

                // Footer
                sb.append("0.5 0.5 0.5 rg\n");
                sb.append("40 45 515 0.5 re f\n");
                sb.append("BT /F2 8 Tf 40 32 Td (Generated on: ").append(new Date().toString())
                  .append(" | MOT Logistics Fleet Operations Report) Tj ET\n");

                byte[] streamBytes = sb.toString().getBytes(StandardCharsets.ISO_8859_1);

                offsets.add(pdfStream.size());
                pdfStream.write(String.format("%d 0 obj\n<< /Type /Page /Parent 2 0 R /MediaB...
                        pageObj, contentObj, font1Obj, font2Obj).getBytes(StandardCharsets.IS...

                offsets.add(pdfStream.size());
                pdfStream.write(String.format("%d 0 obj\n<< /Length %d >>\nstream\n", content...
                pdfStream.write(streamBytes);
                pdfStream.write("\nendstream\nendobj\n".getBytes(StandardCharsets.ISO_8859_1));
            }

            offsets.add(pdfStream.size());
            pdfStream.write(String.format("%d 0 obj\n<< /Type /Font /Subtype /Type1 /BaseFont...
            offsets.add(pdfStream.size());
            pdfStream.write(String.format("%d 0 obj\n<< /Type /Font /Subtype /Type1 /BaseFont...

            int xrefStart = pdfStream.size();
            pdfStream.write(String.format("xref\n0 %d\n0000000000 65535 f \n", offsets.size()...
            for (int i = 1; i < offsets.size(); i++) {
                pdfStream.write(String.format("%010d 00000 n \n", offsets.get(i)).getBytes(St...
            }
            pdfStream.write(String.format("trailer\n<< /Size %d /Root 1 0 R >>\nstartxref\n%d...

            try (FileOutputStream fos = new FileOutputStream(new File(filePath))) {
                pdfStream.writeTo(fos);
            }
            return true;

        } catch (Exception e) {
            System.err.println("InvoiceGenerator.generateTripsSummaryPDF error: " + e.getMess...
            e.printStackTrace();
            return false;
        }
    }

    /**
     * Generate a formatted, print-ready plain text invoice (.txt).
     */
    public static boolean generateTextInvoice(Invoice invoice, List<Trip> trips, String fileP...
        if (!filePath.toLowerCase().endsWith(".txt")) {
            filePath += ".txt";
        }
        ensureDirectoryExists(filePath);
        SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");

        try (PrintWriter writer = new PrintWriter(new OutputStreamWriter(new FileOutputStream...

            writer.println("============================================================");
            writer.println("                      " + SERVICE_NAME.toUpperCase());
            writer.println("           TRANSPORT TRIP & INVOICE MANAGEMENT SYSTEM");
            writer.println("============================================================");
            writer.println();
            writer.printf("Invoice Number  : %s%n", invoice.getInvoiceNumber());
            writer.printf("Invoice Date    : %s%n", sdf.format(invoice.getInvoiceDate()));
            writer.println("------------------------------------------------------------");
            writer.printf("Billed To       : %s%n", invoice.getCompanyName());
            if (invoice.getDriverName() != null && !invoice.getDriverName().isEmpty()) {
                writer.printf("Driver Name     : %s%n", invoice.getDriverName());
            }
            if (invoice.getVehicleNumber() != null && !invoice.getVehicleNumber().isEmpty()) {
                writer.printf("Vehicle Number  : %s%n", invoice.getVehicleNumber());
            }
            writer.printf("Billing Period  : %s%n", getBillingPeriodText(invoice.getBillingMo...
            writer.println("------------------------------------------------------------------...
            writer.println();

            writer.printf("%-5s %-12s %-15s %-15s %-16s %-16s %14s%n",
                    "Sr.No", "Date", "Driver (Created By)", "Vehicle Reg.", "Source", "Destin...
            writer.println("------------------------------------------------------------------...

            double totalAmount = 0;
            int sr = 1;

            if (trips != null) {
                for (Trip trip : trips) {
                    totalAmount += trip.getAmount();
                    writer.printf("%-5d %-12s %-15s %-15s %-16s %-16s %14.2f%n",
```

```
                            sr++,
                            sdf.format(trip.getTripDate()),
                            trip.getDriverName() != null ? trip.getDriverName() : "-",
                            trip.getVehicleNumber() != null ? trip.getVehicleNumber() : "-",
                            trip.getSourceLocation(),
                            trip.getDestination(),
                            trip.getAmount());
                    }
                }

                writer.println("-----------------------------------------------------------------...
                writer.printf("%-5s %-12s %-15s %-15s %-16s %-16s %14d%n",
                        "", "", "", "", "", "Total Trips:", (trips != null ? trips.size() : 0));
                writer.printf("%-5s %-12s %-15s %-15s %-16s %-16s %14.2f%n",
                        "", "", "", "", "", "TOTAL PAYABLE:", totalAmount);
                writer.println("=================================================================...
                writer.println();

                writer.println("  Authorized Signatory                  Client Company Signatory");
                writer.println("  (" + SERVICE_NAME + ")");
                writer.println();
                writer.println("  _____________________          _____________________");
                writer.println("  Signature & Stamp               Signature & Stamp");
                writer.println();
                writer.println("============================================================");
                writer.printf("  Generated Timestamp: %s%n", new Date());
                writer.println("============================================================");

                return true;

            } catch (Exception e) {
                System.err.println("InvoiceGenerator.generateTextInvoice error: " + e.getMessage());
                return false;
            }
        }

        /**
         * Generate an official Single Trip Receipt / Cash Voucher in plain text.
         */
        public static boolean generateTripReceiptText(Trip trip, String filePath) {
            if (!filePath.toLowerCase().endsWith(".txt")) {
                filePath += ".txt";
            }
            ensureDirectoryExists(filePath);
            SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");

            String driverName = trip.getDriverName();
            if ((driverName == null || driverName.trim().isEmpty() || driverName.equals("-")) && ...
                try {
                    com.transport.model.Driver d = new com.transport.dao.DriverDAO().getDriverByI...
                    if (d != null && d.getDriverName() != null && !d.getDriverName().trim().isEmp...
                        driverName = d.getDriverName();
                    }
                } catch (Exception ignored) {}
            }
            if (driverName == null || driverName.trim().isEmpty()) driverName = "Fleet Driver";

            String compName = trip.getCompanyName();
            if ((compName == null || compName.trim().isEmpty() || compName.equals("-")) && trip.g...
                try {
                    com.transport.model.Company c = new com.transport.dao.CompanyDAO().getCompany...
                    if (c != null && c.getCompanyName() != null && !c.getCompanyName().trim().isE...
                        compName = c.getCompanyName();
                    }
                } catch (Exception ignored) {}
            }
            if (compName == null || compName.trim().isEmpty()) compName = "General Client";

            String vehNum = trip.getVehicleNumber();
            if ((vehNum == null || vehNum.trim().isEmpty() || vehNum.equals("-")) && trip.getVehi...
                try {
                    com.transport.model.Vehicle v = new com.transport.dao.VehicleDAO().getVehicle...
                    if (v != null && v.getRegistrationNumber() != null && !v.getRegistrationNumbe...
                        vehNum = v.getRegistrationNumber();
                    }
                } catch (Exception ignored) {}
            }
            if (vehNum == null || vehNum.trim().isEmpty()) vehNum = "Fleet Vehicle";

            try (PrintWriter writer = new PrintWriter(new OutputStreamWriter(new FileOutputStream...
                writer.println("============================================================");
                writer.println("                      " + SERVICE_NAME.toUpperCase());
                writer.println("           OFFICIAL TRIP RECEIPT & DELIVERY VOUCHER");
                writer.println("============================================================");
                writer.printf("Receipt Number : REC-%s%n", trip.getTripCode());
                writer.printf("Trip Date      : %s%n", sdf.format(trip.getTripDate()));
                writer.println("------------------------------------------------------------");
                writer.printf("Client Billed  : %s%n", compName);
                writer.printf("Driver Name    : %s (In-Charge)%n", driverName);
```

```
693 |              writer.printf("Vehicle Number : %s%n", vehNum);
694 |              writer.printf("Pickup (From)  : %s%n", trip.getSourceLocation());
695 |              writer.printf("Drop (To)      : %s%n", trip.getDestination());
696 |              writer.printf("Remarks        : %s%n", trip.getRemarks() != null ? trip.getRemark...
697 |              writer.println("------------------------------------------------------------");
698 |              writer.printf("VOUCHER TOTAL  : INR %.2f%n", trip.getAmount());
699 |              writer.println("============================================================");
700 |              writer.println();
701 |              writer.println("  Driver Signature                    Receiver Signature");
702 |              writer.println();
703 |              writer.println("  _____________________          _____________________");
704 |              writer.println();
705 |              writer.println("============================================================");
706 |              writer.printf("  Generated: %s%n", new Date());
707 |              writer.println("============================================================");
708 |              return true;
709 |          } catch (Exception e) {
710 |              System.err.println("InvoiceGenerator.generateTripReceiptText error: " + e.getMess...
711 |              return false;
712 |          }
713 |      }
714 |
715 |      /**
716 |       * Export a list of trips into a comma-separated (CSV) spreadsheet.
717 |       */
718 |      public static boolean exportTripsToCSV(List<Trip> trips, String filePath) {
719 |          if (!filePath.toLowerCase().endsWith(".csv")) {
720 |              filePath += ".csv";
721 |          }
722 |          ensureDirectoryExists(filePath);
723 |          SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
724 |
725 |          try (PrintWriter writer = new PrintWriter(new OutputStreamWriter(new FileOutputStream...
726 |              writer.println("Trip ID,Trip Code,Date,Client Company,Driver Name,Vehicle Number,...
727 |              if (trips != null) {
728 |                  for (Trip t : trips) {
729 |                      writer.printf("%d,\"%s\",\"%s\",\"%s\",\"%s\",\"%s\",\"%s\",\"%s\",%.2f,\...
730 |                              t.getId(),
731 |                              escapeCsv(t.getTripCode()),
732 |                              sdf.format(t.getTripDate()),
733 |                              escapeCsv(t.getCompanyName()),
734 |                              escapeCsv(t.getDriverName()),
735 |                              escapeCsv(t.getVehicleNumber()),
736 |                              escapeCsv(t.getSourceLocation()),
737 |                              escapeCsv(t.getDestination()),
738 |                              t.getAmount(),
739 |                              escapeCsv(t.getRemarks()));
740 |                  }
741 |              }
742 |              return true;
743 |          } catch (Exception e) {
744 |              System.err.println("InvoiceGenerator.exportTripsToCSV error: " + e.getMessage());
745 |              return false;
746 |          }
747 |      }
748 |
749 |      private static String escapeCsv(String str) {
750 |          if (str == null) return "";
751 |          return str.replace("\"", "\"\"");
752 |      }
753 |
754 |      /**
755 |       * Automatically saves both .pdf and .txt invoice files to the invoices/ directory.
756 |       */
757 |      public static void autoSaveInvoice(Invoice invoice, List<Trip> trips) {
758 |          File invoiceDir = new File("invoices");
759 |          if (!invoiceDir.exists()) invoiceDir.mkdirs();
760 |
761 |          String baseName = "invoices/" + invoice.getInvoiceNumber() + "_MOT_Logistics";
762 |          generatePDF(invoice, trips, baseName + ".pdf");
763 |          generateTextInvoice(invoice, trips, baseName + ".txt");
764 |      }
765 |
766 |      /** Ensures that parent directories exist before creating a file */
767 |      private static void ensureDirectoryExists(String filePath) {
768 |          File f = new File(filePath);
769 |          File parent = f.getParentFile();
770 |          if (parent != null && !parent.exists()) {
771 |              parent.mkdirs();
772 |          }
773 |      }
774 |
775 |      /** Helper to escape parenthesis and backslashes in PDF text strings */
776 |      private static String cleanPdfText(String text) {
777 |          if (text == null) return "";
778 |          return text.replace("\\", "\\\\")
779 |                      .replace("(", "\\(")
780 |                      .replace(")", "\\)")
781 |                      .replace("INR ", "INR ")
```

```
                    .replace("\n", " ")
                    .replace("\r", " ")
                    .replaceAll("[^\\x20-\\x7E]", " ");
    }
}
```

FILE: src/com/transport/util/Validation.java **[UTIL] | 111 lines**

Input Sanitization, Mobile, Email & Vehicle Plate Validation

```
package com.transport.util;

/**
 * Validation.java - Input Validation Utility Class
 *
 * This class contains STATIC methods to validate user inputs.
 * Static means you don't need to create an object to use them.
 * Just call: Validation.isValidMobile("9876543210")
 *
 * KEY CONCEPTS:
 * - Static methods
 * - String methods (isEmpty, matches, length)
 * - Regular expressions (regex) for pattern matching
 */
public class Validation {

    // Private constructor - nobody should create an object of this class
    private Validation() {}

    /**
     * Check if a text field is empty or blank.
     * @param text - the input to check
     * @return true if empty, false if has content
     */
    public static boolean isEmpty(String text) {
        return text == null || text.trim().isEmpty();
    }

    /**
     * Validate a 10-digit Indian mobile number.
     * Must start with 6, 7, 8, or 9.
     * Example: 9876543210 - valid
     * Example: 12345 - invalid
     */
    public static boolean isValidMobile(String mobile) {
        if (isEmpty(mobile)) return false;
        // Regex: starts with [6-9], followed by exactly 9 digits
        return mobile.matches("[6-9][0-9]{9}");
    }

    /**
     * Validate an email address format.
     */
    public static boolean isValidEmail(String email) {
        if (isEmpty(email)) return false;
        return email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
    }

    /**
     * Validate that a value is a valid positive number.
     * Used for trip amount validation.
     */
    public static boolean isValidAmount(String amount) {
        if (isEmpty(amount)) return false;
        try {
            double val = Double.parseDouble(amount);
            return val > 0; // Amount must be greater than 0
        } catch (NumberFormatException e) {
            return false; // Not a number
        }
    }

    /**
     * Validate vehicle registration number format.
     * Example: MH-48-AB-1234
     * Indian format: 2 letters - 2 digits - 2 letters - 4 digits
     */
    public static boolean isValidVehicleNumber(String regNo) {
        if (isEmpty(regNo)) return false;
        // Allow flexible format - just check it's not too short
        return regNo.trim().length() >= 5;
    }

    /**
     * Validate minimum text length.
     * @param text - input text
     * @param minLength - minimum required characters
     */
    public static boolean hasMinLength(String text, int minLength) {
```

```
80 |          if (isEmpty(text)) return false;
81 |          return text.trim().length() >= minLength;
82 |      }
83 |
84 |      /**
85 |       * Check if a username is valid.
86 |       * Must be 4-20 characters, letters/numbers only, no spaces.
87 |       */
88 |      public static boolean isValidUsername(String username) {
89 |          if (isEmpty(username)) return false;
90 |          return username.matches("[a-zA-Z0-9]{4,20}");
91 |      }
92 |
93 |      /**
94 |       * Check if a password meets minimum requirements.
95 |       * Must be at least 4 characters.
96 |       */
97 |      public static boolean isValidPassword(String password) {
98 |          if (isEmpty(password)) return false;
99 |          return password.length() >= 4;
100|      }
101|
102|      /**
103|       * Check if a GST number has valid format.
104|       * GST format: 15 characters
105|       * Example: 27AAAPZ2323J1Z5
106|       */
107|      public static boolean isValidGST(String gst) {
108|          if (isEmpty(gst)) return true; // GST is optional
109|          return gst.trim().length() == 15;
110|      }
111|  }
```

FILE: src/com/transport/ui/TripEntryPanel.java **[UI/WORKFLOW] | 266 lines**

Driver Trip Logging Form & Instant Company Connection Engine

```
1  |  package com.transport.ui;
2  |
3  |  import com.transport.dao.CompanyDAO;
4  |  import com.transport.dao.TripDAO;
5  |  import com.transport.dao.VehicleDAO;
6  |  import com.transport.model.Company;
7  |  import com.transport.model.Driver;
8  |  import com.transport.model.Trip;
9  |  import com.transport.model.Vehicle;
10 |  import com.transport.util.Validation;
11 |
12 |  import javax.swing.*;
13 |  import java.awt.*;
14 |  import java.util.Date;
15 |  import java.util.List;
16 |
17 |  /**
18 |   * TripEntryPanel.java - Real-Time Daily Trip Logging for Drivers
19 |   *
20 |   * Implements Module 5:
21 |   * 1. Automatically generated Trip ID
22 |   * 2. Trip date
23 |   * 3. Company name (dropdown)
24 |   * 4. Driver name (fixed to logged-in driver)
25 |   * 5. Vehicle number (dropdown of driver's vehicles)
26 |   * 6. Source location
27 |   * 7. Destination location
28 |   * 8. Cargo / Container / Goods description (optional)
29 |   * 9. Trip amount
30 |   * 10. Remarks (optional)
31 |   */
32 |  public class TripEntryPanel extends JPanel implements Refreshable {
33 |
34 |      private JTextField         tripCodeField;
35 |      private JSpinner           dateSpinner;
36 |      private JComboBox<Company> companyCombo;
37 |      private JTextField         driverNameField;
38 |      private JComboBox<Vehicle> vehicleCombo;
39 |      private JTextField         sourceField;
40 |      private JTextField         destinationField;
41 |      private JTextField         goodsDescField;
42 |      private JTextField         amountField;
43 |      private JTextField         remarksField;
44 |
45 |      private ModernButton submitBtn;
46 |      private ModernButton clearBtn;
47 |
48 |      private Driver     loggedInDriver;
49 |      private TripDAO    tripDAO    = new TripDAO();
50 |      private CompanyDAO companyDAO = new CompanyDAO();
51 |      private VehicleDAO vehicleDAO = new VehicleDAO();
52 |
```

```
53 |      public TripEntryPanel(Driver driver) {
54 |          this.loggedInDriver = driver;
55 |          setLayout(new BorderLayout(15, 15));
56 |          setBorder(BorderFactory.createEmptyBorder(15, 25, 15, 25));
57 |          setBackground(UITheme.BG_LIGHT);
58 |
59 |          initUI();
60 |      }
61 |
62 |      private void initUI() {
63 |          JPanel formPanel = new JPanel(new GridBagLayout());
64 |          formPanel.setBackground(UITheme.CARD_BG);
65 |          formPanel.setBorder(UITheme.createCardBorder("Log New Daily Transport Trip"));
66 |
67 |          GridBagConstraints gbc = new GridBagConstraints();
68 |          gbc.insets = new Insets(8, 12, 8, 12);
69 |          gbc.fill   = GridBagConstraints.HORIZONTAL;
70 |          gbc.anchor = GridBagConstraints.WEST;
71 |
72 |          tripCodeField = new JTextField(tripDAO.generateTripCode());
73 |          tripCodeField.setEditable(false);
74 |          tripCodeField.setBackground(new Color(241, 245, 249));
75 |          tripCodeField.setFont(UITheme.FONT_BODY_BOLD);
76 |          tripCodeField.setPreferredSize(new Dimension(240, 32));
77 |
78 |          SpinnerDateModel dateModel = new SpinnerDateModel(new Date(), null, null, java.util.C...
79 |          dateSpinner = new JSpinner(dateModel);
80 |          dateSpinner.setEditor(new JSpinner.DateEditor(dateSpinner, "dd/MM/yyyy"));
81 |          dateSpinner.setFont(UITheme.FONT_BODY);
82 |          dateSpinner.setPreferredSize(new Dimension(240, 32));
83 |
84 |          companyCombo = new JComboBox<>();
85 |          companyCombo.setFont(UITheme.FONT_BODY);
86 |          companyCombo.setPreferredSize(new Dimension(240, 32));
87 |          loadCompanies();
88 |
89 |          driverNameField = new JTextField(loggedInDriver.getDriverName());
90 |          driverNameField.setEditable(false);
91 |          driverNameField.setBackground(new Color(241, 245, 249));
92 |          driverNameField.setFont(UITheme.FONT_BODY_BOLD);
93 |          driverNameField.setPreferredSize(new Dimension(240, 32));
94 |
95 |          vehicleCombo = new JComboBox<>();
96 |          vehicleCombo.setFont(UITheme.FONT_BODY);
97 |          vehicleCombo.setPreferredSize(new Dimension(240, 32));
98 |          loadVehicles();
99 |
100 |          sourceField      = new JTextField(20);
101 |          destinationField = new JTextField(20);
102 |          goodsDescField   = new JTextField(20);
103 |          amountField      = new JTextField(20);
104 |          remarksField     = new JTextField(20);
105 |
106 |          styleInputField(sourceField);
107 |          styleInputField(destinationField);
108 |          styleInputField(goodsDescField);
109 |          styleInputField(amountField);
110 |          styleInputField(remarksField);
111 |
112 |          int row = 0;
113 |          addFormRow(formPanel, gbc, row++, "Trip Reference ID:", tripCodeField);
114 |          addFormRow(formPanel, gbc, row++, "Trip Date (dd/mm/yyyy): *", dateSpinner);
115 |          addFormRow(formPanel, gbc, row++, "Select Client Company: *", companyCombo);
116 |          addFormRow(formPanel, gbc, row++, "Driver In-Charge:", driverNameField);
117 |          addFormRow(formPanel, gbc, row++, "Assigned Vehicle Reg.: *", vehicleCombo);
118 |          addFormRow(formPanel, gbc, row++, "Source Location (From): *", sourceField);
119 |          addFormRow(formPanel, gbc, row++, "Destination (To): *", destinationField);
120 |          addFormRow(formPanel, gbc, row++, "Goods / Container Description:", goodsDescField);
121 |          addFormRow(formPanel, gbc, row++, "Trip Fare / Amount (INR ): *", amountField);
122 |          addFormRow(formPanel, gbc, row++, "Remarks / Operational Notes:", remarksField);
123 |
124 |          // Action Buttons
125 |          JPanel btnPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 20, 10));
126 |          btnPanel.setOpaque(false);
127 |
128 |          submitBtn = new ModernButton("[OK]  Submit Trip Entry", UITheme.SUCCESS_GREEN);
129 |          submitBtn.setFont(new Font("Segoe UI", Font.BOLD, 15));
130 |          submitBtn.setPreferredSize(new Dimension(220, 42));
131 |
132 |          clearBtn = new ModernButton("   Clear Fields", UITheme.SECONDARY_GRAY);
133 |          clearBtn.setFont(new Font("Segoe UI", Font.BOLD, 14));
134 |          clearBtn.setPreferredSize(new Dimension(140, 42));
135 |
136 |          submitBtn.addActionListener(e -> submitTrip());
137 |          clearBtn.addActionListener(e  -> clearForm());
138 |
139 |          btnPanel.add(submitBtn);
140 |          btnPanel.add(clearBtn);
141 |
```

```
142 |           JPanel wrapper = new JPanel(new FlowLayout(FlowLayout.CENTER));
143 |           wrapper.setOpaque(false);
144 |           wrapper.add(formPanel);
145 |
146 |           add(new JScrollPane(wrapper), BorderLayout.CENTER);
147 |           add(btnPanel, BorderLayout.SOUTH);
148 |       }
149 |
150 |       private void styleInputField(JTextField field) {
151 |           field.setFont(UITheme.FONT_BODY);
152 |           field.setPreferredSize(new Dimension(240, 32));
153 |       }
154 |
155 |       private void addFormRow(JPanel panel, GridBagConstraints gbc, int row, String labelText, ...
156 |           gbc.gridx = 0; gbc.gridy = row;
157 |           gbc.weightx = 0.35;
158 |           JLabel lbl = new JLabel(labelText);
159 |           lbl.setFont(UITheme.FONT_BODY_BOLD);
160 |           lbl.setForeground(UITheme.TEXT_DARK);
161 |           panel.add(lbl, gbc);
162 |
163 |           gbc.gridx = 1;
164 |           gbc.weightx = 0.65;
165 |           panel.add(field, gbc);
166 |       }
167 |
168 |       private void loadCompanies() {
169 |           companyCombo.removeAllItems();
170 |           List<Company> list = companyDAO.getAllCompanies();
171 |           for (Company c : list) {
172 |               companyCombo.addItem(c);
173 |           }
174 |       }
175 |
176 |       private void loadVehicles() {
177 |           vehicleCombo.removeAllItems();
178 |           // First look for vehicles assigned to this driver
179 |           List<Vehicle> list = vehicleDAO.getVehiclesByDriver(loggedInDriver.getId());
180 |           if (list.isEmpty()) {
181 |               // Fallback to all vehicles in system
182 |               list = vehicleDAO.getAllVehicles();
183 |           }
184 |           for (Vehicle v : list) {
185 |               vehicleCombo.addItem(v);
186 |           }
187 |       }
188 |
189 |       private void submitTrip() {
190 |           if (sourceField.getText().trim().isEmpty() || destinationField.getText().trim().isEmp...
191 |               JOptionPane.showMessageDialog(this, "Source and Destination locations are require...
192 |               return;
193 |           }
194 |
195 |           if (companyCombo.getSelectedItem() == null) {
196 |               JOptionPane.showMessageDialog(this, "Please select a client company from the drop...
197 |               return;
198 |           }
199 |
200 |           if (vehicleCombo.getSelectedItem() == null) {
201 |               JOptionPane.showMessageDialog(this, "Please select or register a vehicle first un...
202 |               return;
203 |           }
204 |
205 |           if (!Validation.isValidAmount(amountField.getText().trim())) {
206 |               JOptionPane.showMessageDialog(this, "Please enter a valid positive trip amount (e...
207 |               return;
208 |           }
209 |
210 |           double amount = Double.parseDouble(amountField.getText().trim());
211 |           Company selectedComp = (Company) companyCombo.getSelectedItem();
212 |           Vehicle selectedVeh  = (Vehicle) vehicleCombo.getSelectedItem();
213 |
214 |           Trip trip = new Trip();
215 |           trip.setTripCode(tripCodeField.getText());
216 |           trip.setTripDate((Date) dateSpinner.getValue());
217 |           trip.setCompanyId(selectedComp.getId());
218 |           trip.setDriverId(loggedInDriver.getId());
219 |           trip.setVehicleId(selectedVeh.getId());
220 |           trip.setSourceLocation(sourceField.getText().trim());
221 |           trip.setDestination(destinationField.getText().trim());
222 |           trip.setGoodsDescription(goodsDescField.getText().trim().isEmpty() ? "General Cargo" ...
223 |           trip.setAmount(amount);
224 |           trip.setTripStatus("Completed");
225 |           trip.setRemarks(remarksField.getText().trim());
226 |
227 |           if (tripDAO.addTrip(trip)) {
228 |               JOptionPane.showMessageDialog(this,
229 |                   "[OK] Transport Trip Recorded & Connected to Company!\n\n" +
230 |                   "Trip Reference ID  : " + trip.getTripCode() + "\n" +
```

```
                "Client Company     : " + selectedComp.getCompanyName() + " (Directly visible...
                "Created By Driver  : " + loggedInDriver.getDriverName() + " (Mobile: " + log...
                "Assigned Vehicle   : " + selectedVeh.getRegistrationNumber() + " (" + select...
                "Operational Route  : " + trip.getSourceLocation() + " -> " + trip.getDestina...
                "Cargo Description  : " + trip.getGoodsDescription() + "\n" +
                "Trip Fare Amount   : INR  " + String.format("%.2f", amount) + "\n\n" +
                "The trip is now live and linked to " + selectedComp.getCompanyName() + "'s t...
                "Trip Submitted & Connected", JOptionPane.INFORMATION_MESSAGE);

            clearForm();
        } else {
            JOptionPane.showMessageDialog(this, "Database error: Failed to record trip entry....
        }
    }

    private void clearForm() {
        tripCodeField.setText(tripDAO.generateTripCode());
        dateSpinner.setValue(new Date());
        if (companyCombo.getItemCount() > 0) companyCombo.setSelectedIndex(0);
        if (vehicleCombo.getItemCount() > 0) vehicleCombo.setSelectedIndex(0);
        sourceField.setText("");
        destinationField.setText("");
        goodsDescField.setText("");
        amountField.setText("");
        remarksField.setText("");
    }

    @Override
    public void refreshData() {
        loadCcompanies();
        loadVehicles();
        if (tripCodeField.getText().trim().isEmpty()) {
            tripCodeField.setText(tripDAO.generateTripCode());
        }
    }
}
```